

ENHANCING SDN NETWORK SECURITY USING GAME THEORY

Prepared by

Nagham Attieh

Bayan Alnassari

Shouq Alkhashman

Supervised by

Dr. Wassim Juneidi

Eng. Mohammad Yamen Hallak

Table of Contents

1	Chapter One: Introduction	7
1.1	Background	8
1.2	Problem Statement	8
1.3	Project Motivation	9
1.4	Aim of the Project	10
1.5	Project Objectives	10
1.6	Literature Basis	11
1.7	Research Gap	14
1.8	Challenges	15
1.9	Practical Applications	15
1.10	Chapter Summary	16
2	Chapter Two: Theoretical Framework	17
2.1	Software-Defined Networking	18
2.2	SDN Architecture	18
2.3	OpenFlow Protocol	20
2.4	Open vSwitch Traffic Mirroring	20
2.5	SDN Controller	21
2.6	DDoS Attacks in SDN	21
2.7	Intrusion Detection Systems	22
2.8	Snort IDS	22
2.9	Snort Alert Forwarding Agent	23
2.10	Game Theory	23
2.11	Attacker–Defender Interaction	23
2.12	Chapter Summary	26
3	Chapter Three: Working Environment	27
3.1	Introduction	28
3.2	Implementation Environment and Software Components	28
3.3	Proposed SDN Security Framework	32
3.4	Practical Implementation Framework	33
3.5	Network Interfaces and VM Layout	35
3.6	Mininet Network Topology Verification	36
3.7	Ryu Controller Initialization	37
3.8	Snort IDS Initialization	38

3.9	Open vSwitch Traffic Mirroring Configuration.....	38
3.10	Snort Alert Forwarding Agent Preparation.....	40
3.11	Chapter Summary	40
4	Chapter Four: Proposed Solution	42
4.1	Introduction	43
4.2	Overall Architecture.....	43
4.3	Observation and Threat Evaluation	44
4.4	Game Theory-Based Decision Model.....	46
4.4.1	Players.....	46
4.4.2	Attacker Strategy	47
4.4.3	Defender Strategy.....	47
4.4.4	Observation Vector	48
4.4.5	Threat Score Calculation	48
4.4.6	Belief Estimation	49
4.4.7	Defender Payoff Matrix.....	50
4.4.8	Expected Utility Calculation	50
4.4.9	Optimal Defense Selection.....	51
4.5	Snort Rules and Alert Forwarding Mechanism	52
4.6	Mitigation Strategy Selection.....	52
4.7	OpenFlow-Based Enforcement	53
4.8	Chapter Summary	54
5	Chapter Five: Experimental Results and Evaluation	55
5.1	Introduction.....	56
5.2	Summary of Experimental Results	56
5.3	Game-Theoretic Decision Progression.....	57
5.4	Snort-Based ICMP Flood Detection.....	57
5.5	Adaptive Defense and Rate-Limiting Decisions	59
5.6	Packet Loss and Connectivity Impact.....	66
5.7	BLOCK Decision and OpenFlow Drop Rule.....	67
5.8	IP Spoofing Attack Result	69
5.9	Evaluation Summary	72
6	Chapter Six: Conclusion.....	74
	References.....	78

List of Tables

Table 1.1 .Summary of the adopted studies supporting the project	13
Table 2.1 Game-Theoretic Mapping Used in the Proposed Project.	24
Table 2.2 Functional role of the core framework components.....	26
Table 3.1 Software Components and Versions.	29
Table 3.2 Implementation Tools and Their Roles.	30
Table 3.3 Measurement Units Used in the Experimental Evaluation.	31
Table 3.4 VM Interfaces Used for Management and Monitoring.	36
Table 4.1 Threat Observation Interpretation Matrix.....	45
Table 4.2 Attacker Strategy Space.	47
Table 4.3 Defender Strategy Space.	48
Table 4.4 Defender Payoff Matrix.	50
Table 4.5 Mitigation Actions Used by the Proposed Framework.	52
Table 4.6 OpenFlow Enforcement Mapping.....	53
Table 5.1 Summary of Experimental Results	56

List of Figures

Figure 2.1 Basic SDN architecture used as the theoretical basis of the project, adapted from.	19
Figure 2.2 Conceptual security workflow of the proposed SDN-based project, adapted from [1] and [4].	25
Figure 3.1 Proposed SDN Security Solution Framework.	32
Figure 3.2 The Proposed Solution Implementation Framework	33
Figure 3.3 Mininet VM Network Interfaces.	35
Figure 3.4 Snort VM Network Interfaces.	35
Figure 3.5 Mininet Two-Switch Topology Verification.	37
Figure 3.6 Ryu Controller Startup and OpenFlow Switch Connection.....	37
Figure 3.7 Snort IDS Initialization on the Monitoring Interface.	38
Figure 3.8 Open vSwitch Ports Connected to Monitoring Interfaces.....	39
Figure 3.9 Open vSwitch Mirroring Configuration for s1 and s2.	39
Figure 3.10 Snort Alert Forwarding Agent Startup.	40
Figure 4.1 Proposed Solution Workflow.....	44
Figure 4.2 Game Theory Model for Optimal Defense Selection.....	46
Figure 4.3 Snort Alert Forwarding Mechanism.....	52
Figure 5.1 ICMP Attack Detection Using Snort IDS in the SDN Environment.	57
Figure 5.2 Dynamic Defense Mechanisms and Suspicious Traffic Rate Limiting in the SDN Environment.	59
Figure 5.3 Real-Time Attack Detection and Threat Assessment Logs in the SDN Environment.	61
Figure 5.4 Dynamic Defense Decision Execution and Mitigation Strategy Enforcement in the SDN Environment.	63
Figure 5.5 Real-Time Monitoring and Adaptive Mitigation Strategy Enforcement during ICMP Flood Attack.	64
Figure 5.6 Impact of ICMP Flood Attack on Connectivity Quality and Packet Loss within the SDN Environment.	66
Figure 5.7 Verification of OpenFlow Drop Rule Installation within the Open vSwitch.	67
Figure 5.8 Detection of IP Spoofing Attack and Dynamic Blocking Enforcement within the SDN Environment.	69
Figure 5.9 : Detection of Suspicious ICMP Traffic by Snort IDS within the SDN Environment.....	71

Abstract

Software-Defined Networking (SDN) provides centralized control and high programmability, which makes network management more flexible and efficient. However, this centralized architecture also introduces security challenges, especially when the controller or network resources are targeted by flooding attacks or spoofed traffic. This project presents an adaptive SDN security framework based on game-theoretic decision making to detect and mitigate suspicious traffic in an emulated SDN environment.

The proposed system integrates Mininet, Open vSwitch, the Ryu SDN Controller, Snort IDS, and an alert forwarding agent. Mininet is used to build the SDN topology, while Open vSwitch forwards traffic and applies mitigation rules through OpenFlow. Network traffic is mirrored from the OpenFlow switches to a separate Snort VM, where suspicious ICMP flooding and spoofing-related traffic are inspected. When Snort detects suspicious activity, the alert forwarding agent sends normalized alerts to the Ryu controller.

The Ryu controller evaluates several observations, including packet rate, Snort alert status, spoofing indicators, packet loss, RTT, and repeated decision rounds. These observations are processed by a game-theoretic decision engine that models the interaction between the attacker and the defender. Based on the estimated threat level and expected defender utility, the system selects one of five mitigation actions: ALLOW, RL_1, RL_2, RL_3, or BLOCK. The rate-limiting actions gradually reduce suspicious traffic, while the BLOCK action installs a high-priority OpenFlow drop rule for severe or persistent attacks.

The experimental results show that the proposed framework can detect ICMP Flood and IP Spoofing attacks, forward Snort alerts to the controller, apply graduated rate-limiting decisions, and block malicious traffic when necessary. The project demonstrates the feasibility of combining SDN programmability, IDS-based detection, and game-theoretic reasoning to provide adaptive network protection in an emulated SDN environment.

Keywords: Software-Defined Networking, Game Theory, Ryu Controller, Snort IDS, OpenFlow, Open vSwitch, ICMP Flood, IP Spoofing, DDoS Mitigation.

1 Chapter One: Introduction

1.1 Background

Software-Defined Networking (SDN) has emerged as a transformative networking paradigm that separates the control plane from the data plane and centralizes network intelligence within a programmable controller. This architectural shift has significantly improved network manageability, scalability, and flexibility, allowing administrators to configure policies, optimize traffic flows, and deploy services more efficiently than in traditional networks. As SDN continues to gain importance in cloud computing, virtualization, and security-aware infrastructures, its programmability has made it a promising platform for building more responsive and adaptive defense mechanisms [1], [2], [3].

Despite these advantages, SDN introduces new security concerns. The controller, as the logical core of the network, becomes a strategic target for adversaries. Among the most serious threats in this context are Distributed Denial-of-Service (DDoS) attacks, which attempt to overwhelm the control layer with malicious traffic, degrade network responsiveness, and disrupt service availability. The literature consistently emphasizes that the same centralization that gives SDN its strength can also become its greatest vulnerability if not properly protected [1], [3], [5].

In recent years, research has increasingly moved beyond static protection methods toward adaptive security frameworks that combine centralized control, intelligent traffic inspection, and dynamic response. This shift is particularly visible in studies that integrate SDN with Intrusion Detection Systems (IDSs), flow-based traffic monitoring, and game-theoretic decision models. These directions are highly relevant to the present project because they support the design of a framework that does not stop at attack detection, but extends toward strategic and context-aware mitigation [1], [4], [5].

1.2 Problem Statement

Although SDN offers centralized control and fine-grained programmability, it also creates a critical security challenge: the controller may act as a single point of failure. When the controller is targeted by DDoS attacks or overwhelmed by abnormal flow requests, the entire network may suffer from latency, packet loss, reduced service quality, or complete disruption. This issue becomes more severe in dynamic environments where legitimate and malicious traffic must be distinguished in real time [1], [2], [3].

Existing SDN security solutions often exhibit one or more limitations. Some approaches are detection-oriented and do not provide a direct and integrated mitigation mechanism. Others achieve strong classification performance but rely on computationally expensive machine learning or deep learning techniques that may increase overhead and reduce suitability for real-time deployment near the controller. In addition, many studies are still evaluated primarily in simulated or controlled environments, with limited validation in realistic or resource-constrained settings [3], [4], [5].

Accordingly, the core problem addressed in this project is the lack of a practical, adaptive, and lightweight SDN security framework that can detect DDoS-related malicious traffic and support appropriate mitigation decisions without imposing excessive burden on the controller or the surrounding infrastructure [1], [3], [5].

1.3 Project Motivation

The motivation for this project is both practical and academic. From a practical perspective, modern programmable networks require more than passive monitoring. A useful security framework must be capable of observing malicious behavior, interpreting it correctly, and responding with a suitable mitigation action. This is particularly important in SDN, where the centralized controller gives the network the ability to enforce policies dynamically and at scale [1], [2].

From an academic perspective, the reviewed literature reveals a persistent gap between detection and mitigation. The SEC-SDN study demonstrates the value of integrating Snort and game-theoretic reasoning into an SDN framework for adaptive DDoS defense. The hybrid mitigation study goes further by stating explicitly that detection alone is not adequate, and that mitigation should be informed by adaptive, strategic reasoning rather than fixed thresholds alone. These findings strongly support the conceptual direction of the current project [1], [4].

The project is also motivated by the growing need for efficient and deployable solutions. Recent work on resource-aware DDoS detection in SDN has shown that high predictive performance must be balanced with memory usage, training time, inference latency, and controller stability. Therefore, a project that combines SDN programmability, practical inspection tools, and adaptive decision-making can offer both academic value and implementation relevance [3], [5].

1.4 Aim of the Project

The main aim of this project is to enhance SDN network security by designing an adaptive DDoS-oriented defense framework that integrates the Ryu Controller, OpenFlow, Open vSwitch traffic mirroring, Snort IDS, and a game-theoretic decision engine. The proposed framework focuses on detecting and mitigating distributed ICMP flooding behavior and spoofing-related traffic in an emulated SDN environment. Instead of relying on a fixed threshold, the framework evaluates multiple indicators, including packet rate, Snort alert status, packet loss, RTT, spoofing indicators, and the number of suspicious sources, in order to select a proportional mitigation action.

1.5 Project Objectives

The project seeks to achieve the following objectives:

1. To examine the security weaknesses of SDN environments, especially controller exposure to DDoS-related traffic.
2. To build an emulated SDN topology using Mininet with two OpenFlow switches and multiple hosts.
3. To configure Open vSwitch mirroring so that traffic from each switch can be inspected by a dedicated Snort VM.
4. To use Snort IDS to detect suspicious ICMP flooding and spoofing-related traffic patterns.
5. To implement an alert forwarding agent that transfers normalized Snort alerts from the Snort VM to the Mininet/Ryu controller VM.
6. To design a game-theoretic decision engine that evaluates observed traffic behavior and selects one of the following actions: ALLOW, RL_1, RL_2, RL_3, or BLOCK.
7. To apply mitigation through OpenFlow-based rate limiting and drop rules.
8. To evaluate the proposed framework using normal traffic, single-source ICMP flood traffic, multi-source DDoS-oriented ICMP flood scenarios, and spoofed ICMP traffic, considering indicators such as packet rate, Snort alerts, attacker count, RTT, packet loss, and mitigation response.

1.6 Literature Basis

The project is supported by a set of complementary studies that shape its conceptual and methodological direction.

The primary and most directly relevant study is the SEC-SDN work by Razvan and Mitica. This study proposes a Security-Enhanced SDN framework integrated with Snort and game-theoretic modeling to improve resilience against DDoS attacks. Its direct relevance to the current project lies in the fact that it combines centralized SDN control, intrusion detection, strategic defense logic, and adaptive mitigation within one integrated security framework. It therefore serves as the strongest conceptual foundation for the present work [1].

The study by Gupta, Tanwar, and Badotra is highly relevant from the practical controller-security perspective. It proposes an early DDoS detection model for SDN using sFlow-based monitoring, integrates with major SDN controllers, and evaluates detection using Mininet-based scenarios and metrics such as detection time, RTT, and packet loss. This makes it particularly valuable in supporting the controller-oriented and implementation-aware side of the project [2].

The systematic review by Wainwright et al. plays a central role in supporting the project's research justification. It shows that although many SDN-based DDoS solutions have been proposed, the literature remains dominated by simulation-heavy validation, and mitigation is rarely evaluated together with detection. It also highlights the importance of lightweight, edge-aware, and resource-conscious approaches, especially in constrained environments [3].

The hybrid approach by Kachavimath and Narayan provides strong support for the project's adaptive security direction. It argues that conventional fixed-rule approaches are insufficient for dynamic SDN traffic, and it presents a framework that combines feature selection, unsupervised clustering, posterior-probability reputation scoring, and game-theoretic mitigation. This study is especially useful because it directly supports the idea of linking detection to adaptive defense rather than treating them as separate stages [4].

The resource-aware ensemble framework by Trigui et al. contributes a modern perspective on SDN security design. It emphasizes that successful detection systems should be not only accurate but also efficient, generalizable, and feasible for deployment. Its focus on resource

profiling, latency, memory usage, and controller-adjacent deployment strongly reinforces the need for practical and lightweight design decisions in the present project [5].

The study by Javadpour et al. provides secondary support for the game-theoretic dimension of the project. Although its primary focus is malicious-node and botnet-oriented modeling rather than controller flooding specifically, it demonstrates that strategic interaction between attackers and defenders can be modeled in a meaningful way in network security settings, and that SDN environments can benefit from such structured adversarial reasoning [6].

Finally, the Advanced EDoS Eye study by Chowdhury et al. offers peripheral but still useful support to the project. It is not an SDN-DDoS study in the strict sense, since it focuses on EDoS in cloud systems, but it is important because it argues that static games are impractical for real attacker–defender conflicts and that dynamic game-based decision modules can improve threshold selection, response quality, and QoS preservation. This supports the broader theoretical argument in favor of dynamic decision-making [7].

The adopted studies support the framework from complementary perspectives, including SDN-based DDoS detection, adaptive mitigation, resource-aware design, and game-theoretic decision making, as summarized in Table 1.1.

Table 1.1 .Summary of the adopted studies supporting the project

Ref.	Study title	Year	Main focus	Contribution to the project
[1]	Enhancing network security through integration of game theory in software-defined networking framework	2025	SEC-SDN framework integrating SDN, Snort, and game theory for adaptive DDoS mitigation	The closest and primary reference for the proposed project design
[2]	A novel sFlow based DDoS detection model in software defined networking	2024	Early DDoS detection in SDN controllers using sFlow with practical controller-based evaluation	Supports the practical monitoring and SDN-controller detection perspective
[3]	Software-Defined Networking Security Detection Strategies and Their Limitations with a Focus on Distributed Denial-of-Service for Small to Medium-Sized Enterprises	2025	Systematic review of SDN-based DDoS detection and mitigation with emphasis on lightweight solutions	Strongly supports the research gap and the need for practical integrated solutions
[4]	Hybrid Approach for Detection and Mitigation of DDoS Attacks Using Multi-feature Selection, Unsupervised Learning, and Game Theory	2025	Adaptive detection and mitigation using feature selection, clustering, and game theory	Supports the detect-to-mitigate logic and adaptive strategic defense
[5]	Resource-Aware Ensemble Framework for DDoS Detection in SDN via Novel Game-Theoretic Feature Selection	2026	Resource-aware SDN DDoS detection with game-theoretic feature selection	Supports efficiency, deployment feasibility, and low-overhead design
[6]	Detecting malicious nodes using game theory and reinforcement learning in software-defined networks	2025	Game-theoretic and learning-based modeling of adversarial network behavior	Supports attacker-defender modeling and adaptive reasoning in SDN security
[7]	Advanced EDoS eye: A pragmatic model to mitigate economic denial of sustainability attack in cloud system applying dynamic game-based decision module	2026	Dynamic game-based decision module for adaptive defense and threshold selection	Supports the theoretical argument for dynamic game-driven response logic

1.7 Research Gap

The reviewed literature reveals several important gaps that justify the present project.

First, many SDN-DDoS solutions are still validated mainly in simulations or controlled testbeds. While such environments are useful for initial evaluation, they do not fully address deployment realism, controller stability, or operational trade-offs in practical settings. This creates uncertainty regarding how well many proposed methods would perform under realistic constraints [3].

Second, much of the literature emphasizes attack detection more than end-to-end mitigation. Some studies achieve promising results in identifying malicious traffic, but the integration between inspection, controller decision-making, and active mitigation remains limited. The lack of tightly coupled detect-to-mitigate pipelines represents a meaningful gap, particularly for SDN where the controller is capable of acting immediately after detection [3], [4].

Third, several modern detection approaches rely on computationally expensive architectures. Although these methods may produce excellent classification results, they are not always suitable for controller-adjacent deployment where latency, memory, and inference efficiency are critical. This highlights the need for security designs that combine intelligence with operational feasibility [3], [5].

Therefore, the present project addresses a clear gap by aiming to build a framework that combines SDN-based control, practical traffic inspection, and game-theoretic mitigation logic in a form that is both conceptually integrated and implementable in an academic test environment [1], [4], [5].

Based on this gap, the present project focuses on building a closed-loop experimental workflow in which mirrored traffic is inspected by Snort, alerts are forwarded to the Ryu controller, the game-theoretic engine evaluates the attack context, and OpenFlow rules are installed to apply graduated mitigation. This makes the project different from purely detection-oriented approaches because it connects observation, detection, decision-making, and mitigation in one practical SDN workflow.

1.8 Challenges

The project is associated with several technical and research challenges.

One major challenge is processing latency. Because SDN depends on timely controller decisions, any delay introduced by inspection, analysis, or mitigation logic may reduce the effectiveness of the defense mechanism. A slow response can allow malicious traffic to influence the network before countermeasures take effect [5].

Another challenge is false positives. IDS-based or anomaly-based systems may occasionally classify legitimate traffic as malicious, especially when network traffic is bursty or highly dynamic. In practical environments, excessive false alarms may result in unnecessary blocking, degraded quality of service, and reduced trust in the defense mechanism [1], [3].

A third challenge is resource overhead. Controller-adjacent security systems must be designed carefully so that they do not consume excessive CPU, memory, or bandwidth. Recent literature shows that even highly accurate systems may be impractical if they impose unacceptable overhead on real-time SDN operations [3], [5].

A fourth challenge is dynamic strategy selection. Detecting suspicious traffic is only the first stage. The framework must still choose how to respond, whether by blocking, rate limiting, rerouting, or continued observation. This decision cannot always rely on static thresholds alone, which is why game theory becomes especially valuable but also intellectually demanding in this project [4], [7].

1.9 Practical Applications

The proposed project has practical relevance in both academic and operational contexts.

In enterprise and cloud-oriented SDN environments, an adaptive framework that links traffic inspection with controller-based response may improve resilience against DDoS attacks by reducing reaction time and helping preserve service continuity. Such integration can contribute to maintaining network availability and protecting the controller from overload under hostile traffic conditions [1], [5].

The project also has strong educational and experimental value. Because it can be developed in an emulated SDN environment using tools such as Mininet, Ryu, Snort, and attack-generation utilities, it offers a practical platform for studying SDN security, IDS integration, DDoS response logic, and attacker–defender strategy modelling [1], [2], [4].

Although the project is implemented in an emulated academic environment rather than a production network, it provides a practical proof-of-concept for studying how SDN programmability, IDS inspection, and game-theoretic reasoning can be combined to support adaptive security decisions.

1.10 Chapter Summary

This chapter introduced the background and significance of the project and explained why SDN security remains a critical issue in modern programmable networks. It clarified the problem of controller vulnerability under DDoS attacks, justified the need for adaptive and efficient security mechanisms, and identified the major studies that support the project from conceptual, methodological, and practical perspectives. It also highlighted the research gap, the major technical challenges, and the practical relevance of the proposed work.

2 Chapter Two: Theoretical Framework

This chapter presents the theoretical foundation of the project. It focuses only on the concepts, technologies, and security principles that are directly used in the design of the proposed framework. In particular, the chapter explains Software-Defined Networking, SDN architecture, OpenFlow, the SDN controller, DDoS attacks in SDN environments, intrusion detection systems, Snort IDS, and the role of game theory in attacker–defender decision making. This theoretical basis is essential because the proposed solution depends on the integration of centralized SDN control, traffic inspection, and adaptive mitigation logic [1], [4].

2.1 Software-Defined Networking

Software-Defined Networking (SDN) is a modern networking model that separates the control plane from the data plane and places control intelligence within a centralized, programmable controller. This separation allows administrators to manage flows, enforce policies, and adapt network behavior more flexibly than in traditional device-centric architectures. As a result, SDN has become an important foundation for cloud systems, network virtualization, traffic engineering, and security-aware infrastructures [1], [2], [3].

In the context of this project, SDN is important not only because it improves manageability, but because it enables adaptive defense. Since the controller maintains a broad view of the network and can dynamically update forwarding behavior, it provides the technical basis for moving from passive observation toward active and programmable mitigation. At the same time, this centralization creates a strategic security exposure, making controller protection a central concern in the present work [1], [3], [5].

2.2 SDN Architecture

The SDN architecture is commonly described as a three-layer model consisting of the application layer, the control layer, and the infrastructure layer. The application layer includes

monitoring, policy, and security services. The control layer contains the SDN controller, which acts as the logical decision-making core of the network. The infrastructure layer contains forwarding devices such as OpenFlow switches that execute the rules installed by the controller [1], [2].

This architectural separation is fundamental to the project because it enables distinct but coordinated roles. Traffic forwarding is handled in the infrastructure layer, while analysis, policy interpretation, and strategic decision-making can be managed through the upper layers. This makes SDN especially suitable for structured security workflows in which suspicious traffic is observed, evaluated, and handled through controller-enforced actions [1], [3].

The layered structure of SDN consists of the application layer, the control layer, and the infrastructure layer, where centralized control enables programmable and security-aware traffic management, as illustrated in Figure 1 [1].

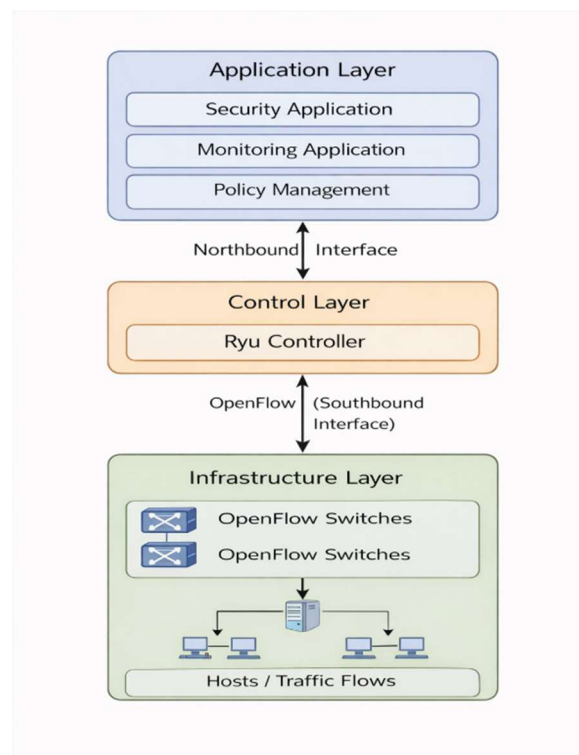


Figure 2.1 Basic SDN architecture used as the theoretical basis of the project, adapted from.

2.3 OpenFlow Protocol

OpenFlow is one of the most important southbound protocols in SDN because it defines how the controller communicates with forwarding devices. Through OpenFlow, the controller can install, modify, and remove flow rules dynamically, thereby controlling how packets are treated at the switch level [1].

In practical terms, this makes OpenFlow a key enabling mechanism for the proposed project. Any controller-based mitigation decision, such as dropping malicious flows, rerouting suspicious traffic, or applying new forwarding behavior, must ultimately be translated into switch rules. Therefore, OpenFlow is not merely a background concept; it is one of the operational foundations that make adaptive mitigation possible [1], [3].

In this project, OpenFlow is not used only for normal forwarding; it is also used as the enforcement mechanism for mitigation decisions. Rate-limiting decisions are implemented through OpenFlow meter-based rules, while severe or persistent attacks are handled using high-priority drop rules. Therefore, OpenFlow represents the link between the game-theoretic decision made by the controller and the actual mitigation action applied inside the data plane.

2.4 Open vSwitch Traffic Mirroring

Open vSwitch traffic mirroring is used to create a copy of selected network traffic and forward it to a monitoring interface without changing the original forwarding path. In this project, mirroring is essential because each OpenFlow switch sends a copy of its observed traffic to the Snort VM for inspection. This allows Snort to analyze suspicious traffic while the SDN controller continues to manage forwarding and mitigation decisions.

The mirroring mechanism supports the separation between traffic forwarding and traffic inspection. The switches remain responsible for forwarding packets between hosts, while the Snort VM acts as an external sensor that observes copied traffic. This design improves realism because the IDS is not placed directly inside the controller logic; instead, it receives mirrored traffic similarly to a monitoring sensor in real network environments.

2.5 SDN Controller

The SDN controller is the central control entity that manages traffic intelligence, enforces policies, and maintains a global network view. It is often regarded as the most critical component in SDN because it coordinates the interaction between applications and forwarding devices and turns centralized policy into real network behavior [1].

In this project, the selected controller is **Ryu**, an open-source SDN controller written in Python and widely used in academic and experimental environments. The SEC-SDN study describes Ryu as well aligned with SDN architecture, with a role that connects upper-layer applications to lower-layer OpenFlow-based switching behavior. This makes it a suitable platform for implementing controller-based logic in a graduation project environment [1].

However, the controller is also the component most exposed to flooding-based disruption. When malicious traffic generates excessive flow requests or burdens the control path, the controller may experience overload, delay, or reduced service quality. For this reason, many recent SDN security studies emphasize not only detection, but also controller-aware and resource-efficient design [3], [5].

2.6 DDoS Attacks in SDN

A Distributed Denial-of-Service (DDoS) attack is a network attack in which multiple malicious or compromised sources flood a target with abnormal traffic to exhaust its resources or degrade its availability. In SDN environments, such attacks are especially dangerous because the centralized controller can be impacted directly or indirectly through malicious flow activity [2], [3], [5].

The literature shows that DDoS attacks in SDN may manifest as controller flooding, Packet-In storms, or malicious traffic patterns that consume network and controller resources. The SEC-SDN study focuses on strengthening resilience against DDoS within an SDN setting, while the sFlow-based study demonstrates controller-oriented early detection under practical emulation scenarios. These directions strongly justify selecting DDoS as the primary threat model of the project [1], [2].

In addition to flooding behavior, IP spoofing is relevant to this project because an attacker may falsify the source IP address to hide its identity or bypass simple source-based filtering. In an SDN environment, spoofing can be detected by comparing the observed source IP address with the expected host identity or network location. When spoofing is combined with ICMP flooding, the attack becomes more critical because the traffic is both high-volume and identity-manipulated.

2.7 Intrusion Detection Systems

An Intrusion Detection System (IDS) is a monitoring mechanism that analyzes traffic or system behavior in order to identify malicious activity, suspicious patterns, or policy violations. IDS solutions may rely on signatures, anomalies, behavioral analysis, or hybrid methods depending on the design goal and threat model [1], [6].

In SDN environments, IDS becomes especially valuable because detection can be connected directly to control logic. Rather than stopping at alert generation, an IDS integrated with the controller can trigger real enforcement actions such as blocking, throttling, or redirecting suspicious traffic. This transformation from passive monitoring to actionable defense is one of the key reasons IDS occupies a central place in the proposed project [1], [3].

2.8 Snort IDS

Snort is a widely used open-source intrusion detection system that performs rule-based inspection of network traffic and generates alerts when suspicious signatures or patterns are detected. In the SEC-SDN framework, Snort is integrated into the SDN environment as an active monitoring component that contributes to real-time threat detection and supports policy-driven response [1].

This role makes Snort highly relevant to the current project. In the proposed framework, Snort represents the inspection layer that examines observed or mirrored traffic and provides the evidence needed for controller-side action. The importance of Snort here is not limited to detection alone; rather, it forms part of a broader security workflow in which traffic inspection is linked to adaptive mitigation [1].

2.9 Snort Alert Forwarding Agent

The Snort alert forwarding agent acts as an integration layer between the Snort VM and the Ryu controller VM. Its role is to monitor Snort alert log files, extract relevant attack indicators such as source IP, destination IP, protocol, and alert type, and forward a normalized alert message to the controller. This component is important because Snort and the controller run on separate virtual machines in the project environment.

By using an alert forwarding agent, the framework separates detection from decision-making. Snort is responsible for traffic inspection and alert generation, while the Ryu controller is responsible for evaluating the alert together with flow observations and applying the appropriate mitigation action.

2.10 Game Theory

Game theory is a mathematical and strategic framework used to analyze interactions between rational entities whose decisions affect one another. In cybersecurity, game theory is frequently used to model the relationship between attackers and defenders, especially in settings where both sides adapt their behavior over time [1], [4], [6].

Its relevance to this project lies in the fact that SDN security is not simply a classification problem. Once malicious traffic is detected, the defender must still decide how to respond in a way that balances protection, service continuity, and operational cost. The hybrid mitigation study explicitly highlights that game theory can help the controller adapt mitigation policies dynamically. The Advanced EDoS Eye study reinforces the same principle by arguing that static game models are impractical in real attacker–defender conflicts and that dynamic strategies are better suited to changing adversarial behavior [4], [7].

2.11 Attacker–Defender Interaction

The attacker–defender model describes the strategic conflict between an adversary attempting to disrupt the network and a defense system attempting to preserve service availability and minimize damage. In real network environments, attackers may vary their traffic intensity, timing, or methods, while defenders must decide whether to observe, challenge, rate limit, block, or reroute traffic [4], [6], [7].

This interaction is central to the theoretical logic of the project. The malicious-node study shows that game-theoretic reasoning can model hidden, deceptive, and adaptive adversarial behavior in network security problems. Although its direct focus is botnet-oriented rather than controller-flooding oriented, it still strengthens the argument that attacker–defender reasoning is a valuable conceptual tool for building smarter network defenses [6].

In the present project, attacker–defender interaction forms the bridge between detection and mitigation. After traffic is inspected and suspicious activity is identified, the framework still needs to choose an appropriate response. This decision should depend on context, severity, and expected impact, which is precisely why game-theoretic reasoning is incorporated into the project’s conceptual design [1], [4], [7].

The overall security logic of the framework moves from traffic observation to inspection, controller-side analysis, strategic decision making, and mitigation, as illustrated in Figure 2.2 [1], [4].

To clarify how game theory is applied in the proposed SDN security framework, Table 2.1 maps the main game-theoretic elements to their corresponding components in this project.

Table 2.1 Game-Theoretic Mapping Used in the Proposed Project.

Game Element	Project Mapping
Attacker	A host generating suspicious or malicious traffic, such as ICMP Flood or Spoofed ICMP Flood.
Defender	The Ryu controller supported by the game-theoretic decision engine.
Attacker Strategies	Normal traffic, probing behavior, high-rate ICMP flooding, spoofed traffic, and persistent attack behavior.
Defender Strategies	ALLOW, RL_1, RL_2, RL_3, and BLOCK.
Observations	Packet rate, Snort alert, spoofing indicator, packet loss, RTT, and repeated decision rounds.
Defender Objective	Reduce the impact of the attack while preserving legitimate traffic as much as possible.
Attacker Objective	Maximize network disruption and avoid detection or mitigation.
Payoff Logic	Stronger and more persistent attacks increase the defender’s incentive to apply stricter mitigation actions.

This mapping shows that the attacker is represented by a host generating suspicious or malicious traffic, while the defender is represented by the Ryu controller supported by the game-theoretic decision engine. The controller evaluates observations such as packet rate, Snort alerts, spoofing indicators, packet loss, and RTT in repeated decision rounds. Based on these observations, the defender selects an adaptive response, starting from ALLOW and escalating through rate-limiting levels until BLOCK is applied when the attack becomes severe or persistent.

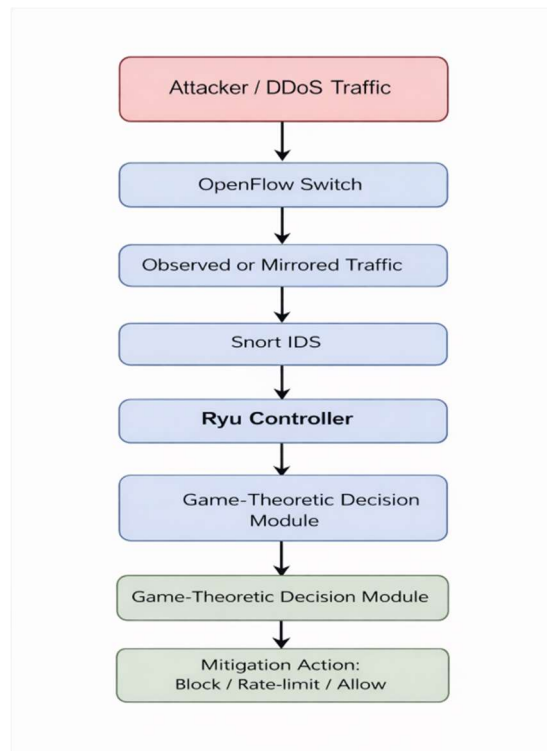


Figure 2.2 Conceptual security workflow of the proposed SDN-based project, adapted from [1] and [4].

The functional role of each major component within the framework is clarified in Table 2.2.

Table 2.2 Functional role of the core framework components.

Component	Function	Relevance to the project
SDN	Provides centralized and programmable network control	Enables dynamic traffic management and security enforcement
OpenFlow	Connects controller decisions to switch rules	Allows flow-based mitigation actions
Ryu Controller	Acts as the main control entity	Applies policies and mitigation logic
Snort IDS	Inspects traffic and generates alerts	Detects suspicious or malicious traffic
Game Theory	Models attacker–defender interaction	Supports adaptive defense selection
DDoS Model	Attack Represents the main threat scenario	Provides the practical security use case of the project

2.12 Chapter Summary

This chapter presented the theoretical foundation of the project by focusing on the core concepts and technologies directly related to its design. It explained Software-Defined Networking, the layered SDN architecture, the role of OpenFlow, the importance of the SDN controller, the nature of DDoS attacks in SDN, the role of IDSs, the function of Snort, and the relevance of game theory in attacker–defender interaction.

3 Chapter Three: Working Environment

3.1 Introduction

This chapter presents the practical implementation environment and system architecture used to develop the proposed SDN security framework. The proposed framework integrates Mininet, Open vSwitch, the Ryu SDN controller, Snort IDS, and a game-theoretic decision engine to provide adaptive attack detection and mitigation within a software-defined networking environment.

The chapter explains the interaction between the data plane, control plane, and intrusion detection components, including traffic mirroring, alert forwarding, and dynamic mitigation enforcement. In addition, it describes how the controller collects network observations such as packet rate, packet loss, RTT, and Snort alerts in order to make adaptive defense decisions based on game-theoretic strategies.

Furthermore, this chapter presents the software environment, implementation tools, network topology, communication architecture, and the integration workflow between the detection and mitigation modules used throughout the experimental setup.

3.2 Implementation Environment and Software Components

The proposed framework was implemented using a collection of virtualization, SDN, traffic analysis, and intrusion detection tools. Particular attention was given to selecting lightweight, stable, and research-oriented software versions to ensure compatibility and efficient interaction between the SDN controller, the IDS engine, and the network emulation environment. Table 3.1 summarizes the main software components used in the implementation phase, including their selected versions, justification, and relevance to the proposed framework.

Table 3.1 Software Components and Versions.

Component	Version	Why This Version Was Selected	Relevance
Ubuntu	Ubuntu 22.04.5 LTS	Stable Linux base for VM deployment and compatibility with SDN tools	Hosts Mininet / Snort / Ryu
Python	Python 3.10.12	Required for Ryu applications and custom automation scripts	Controller and agent logic
Mininet	2.3.0	Lightweight SDN emulation platform widely used in academic research	Topology creation
Open vSwitch	2.17.9	Provides OpenFlow switching, traffic forwarding, and mirroring capabilities	Data-plane switching
Ryu	4.34	Python-based and lightweight SDN controller suitable for research environments	Controller logic
OpenFlow	1.3	Supported by Open vSwitch and suitable for flow and meter enforcement	Enforcement channel
Snort	3.1.5.0	Modern modular IDS with Lua-based configuration and efficient packet inspection	Detection layer
hping3	3.0.0-alpha-2	Practical ICMP flood generation and traffic simulation	Attack simulation
Scapy	2.4.4	Enables custom packet crafting for spoofing and security testing	Spoofing scenario

Among the selected components, Snort 3.1.5.0 played a critical role in the proposed framework due to its modular architecture, efficient packet inspection engine, and Lua-based configuration capabilities. Compared to older IDS versions, Snort 3 provides improved performance, better rule management, and enhanced support for modern network traffic analysis, making it suitable for integration with SDN environments.

Similarly, Open vSwitch 2.17.9 and OpenFlow 1.3 were selected because they support flow-based forwarding and meter enforcement, which are essential for implementing the adaptive rate-limiting strategy proposed in this work. In addition, Ryu 4.34 was chosen as the SDN controller because of its lightweight Python-based architecture and ease of integration with custom security logic and game-theoretic decision mechanisms.

Table 3.2 Implementation Tools and Their Roles.

Component	Tool / Technology	Role in the Project
Network Emulator	Mininet	Builds the virtual SDN topology with hosts and switches
Virtual Switch	Open vSwitch	Provides OpenFlow-based switching and traffic mirroring
SDN Controller	Ryu Controller	Manages flows and applies mitigation decisions
IDS	Snort IDS	Inspects mirrored traffic and generates alerts
Alert Integration	Snort Alert Forwarding Agent	Sends Snort alerts from the Snort VM to the controller VM
Attack Tool	hping3 / Scapy	Generates ICMP Flood and spoofed traffic
Decision Module	Game-Theoretic Engine	Selects ALLOW, RL_1, RL_2, RL_3, or BLOCK

The interaction between the implemented components forms a complete SDN security workflow. Mininet emulates the virtual network topology, while Open vSwitch performs OpenFlow-based packet forwarding inside the data plane. The Ryu controller operates within the control plane and continuously monitors network behavior through collected observations and forwarded alerts. To support intrusion detection, mirrored traffic copies are forwarded from the Open vSwitch bridge to the Snort virtual machine using the port mirroring mechanism. Snort analyzes the mirrored traffic and generates alerts whenever suspicious behavior is detected. A custom Python-based forwarding agent then transfers the generated alerts to the SDN controller through a dedicated management channel between the Snort VM and the controller VM. Based on the collected network observations, including packet rate, RTT variation, packet loss, and Snort alerts, the game-theoretic decision engine dynamically selects the most suitable mitigation strategy. The selected strategy is then enforced using OpenFlow meters and flow rules.

To make the experimental setup measurable and reproducible, the project defines a set of measurement units for the main observations used during traffic evaluation and mitigation decision-making. These units are important because the proposed framework does not rely only on descriptive observations. Instead, it evaluates measurable indicators such as packet rate, packet loss, RTT, Snort alert status, attacker count, and mitigation rate. Table 3.3 summarizes the main measurement units used throughout the experimental evaluation.

Table 3.3 Measurement Units Used in the Experimental Evaluation.

Measurement / Metric	Symbol or Field Name	Unit / Type	Source of Measurement	Purpose in the Project
Packet Rate	packet_rate_pps	packets per second (pps)	Ryu controller / flow monitoring	Measures traffic intensity and helps identify flooding behavior.
Packet Loss	packet_loss_pct	Percentage (%)	Ping / connectivity test	Measures the effect of the attack on communication quality.
Round Trip Time	rtt_ms	Milliseconds (ms)	Ping latency result	Measures network delay caused by congestion or flooding.
Snort Alert Status	snort_alert	Binary value: 0 or 1	Snort IDS alert log	Indicates whether Snort detected suspicious traffic.
Attacker Count	attacker_count	Number of suspicious sources	Controller observation / traffic source analysis	Distinguishes single-source DoS from multi-source DDoS behavior.
Mitigation Rate	rate_kbps	Kilobits per second (kbps)	OpenFlow meter rule	Defines the rate-limiting level applied by RL_1, RL_2, or RL_3.
Attack Duration	duration_sec	Seconds (s)	Attack command / experiment setup	Defines how long the attack traffic is generated during testing.
Decision Round	round	Integer count	Game-theoretic engine log	Tracks repeated evaluation cycles during the attack.
Source IP Address	src_ip	IPv4 address	Snort / Ryu log	Identifies the traffic source or attacker host.
Destination IP Address	dst_ip	IPv4 address	Snort / Ryu log	Identifies the victim host targeted by the traffic.
Spoofing Status	spoofed	Boolean value: True / False	Controller validation / packet analysis	Indicates whether the source identity appears manipulated.

Measurement / Metric	Symbol or Field Name	Unit / Type	Source of Measurement	Purpose in the Project
OpenFlow Action	action	ALLOW / RL_1 / RL_2 / RL_3 / BLOCK	Ryu controller decision	Represents the selected mitigation response.

The measurement units in Table 3.3 clarify how each experimental indicator is represented before being passed to the decision engine. For example, packet rate is expressed in packets per second, packet loss is expressed as a percentage, RTT is expressed in milliseconds, and rate-limiting actions are expressed in kilobits per second. In addition, attacker_count is included to support the DDoS-oriented scenario, where multiple suspicious hosts may target the same victim. This makes the evaluation clearer because each observation has a defined source, unit, and purpose.

3.3 Proposed SDN Security Framework

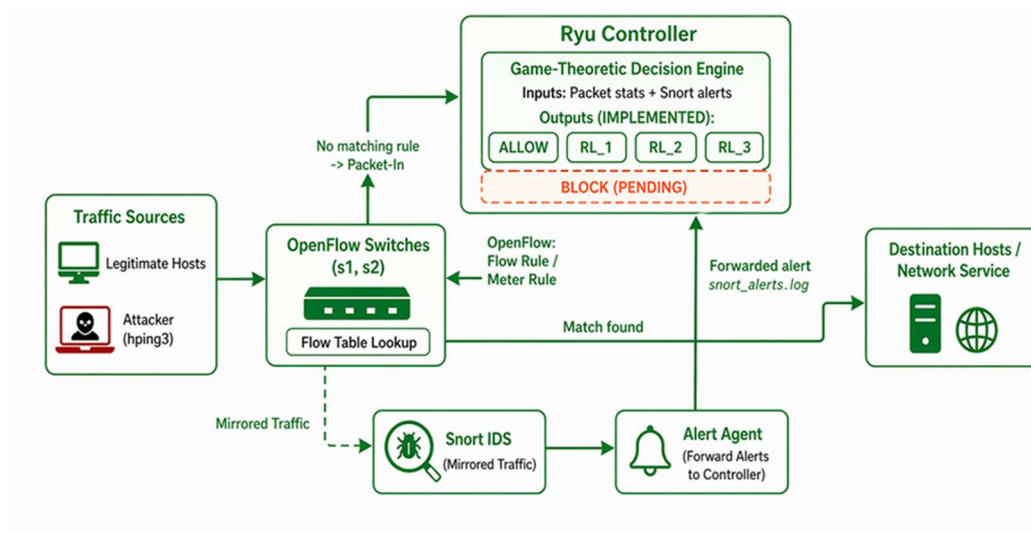


Figure 3.1 Proposed SDN Security Solution Framework.

Figure 3.1 illustrates the general framework of the proposed security solution within the Software-Defined Networking (SDN) environment. The diagram shows how network traffic is generated from traffic sources, including legitimate hosts and an attacker host, and then forwarded toward the OpenFlow switches such as s1 and s2.

The switches forward normal traffic between hosts, while also sending a copied version of the traffic to the Snort IDS through the Traffic Mirroring mechanism. Snort analyzes network packets and detects suspicious activities such as ICMP Flood attacks or spoofing attempts.

When a threat is detected, alerts are sent to the Alert Agent, which forwards them to the Ryu Controller. Inside the controller, the Game Theory-based decision engine analyzes the alerts and the current network state in order to select the most suitable defensive action.

The possible actions include allowing normal traffic (ALLOW), applying different levels of rate limiting (RL_1, RL_2, RL_3), or blocking malicious traffic in severe cases. After selecting the decision, the controller sends OpenFlow rules to the switches to enforce the selected action within the network.

This figure represents the complete workflow of the system, starting from traffic monitoring, attack detection, alert forwarding, decision-making, and finally mitigation or blocking enforcement.

3.4 Practical Implementation Framework

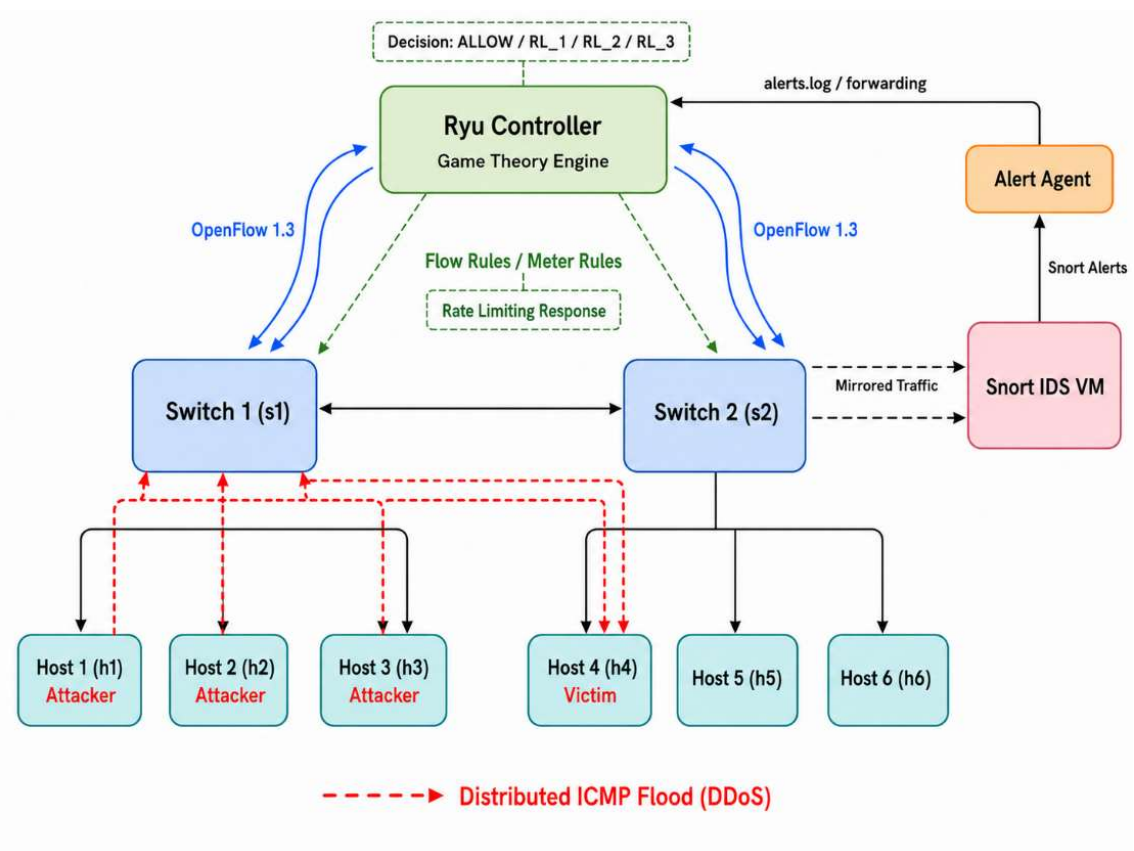


Figure 3.2 The Proposed Solution Implementation Framework

Figure 3.2 illustrates the practical implementation framework of the proposed security solution within the Software-Defined Networking (SDN) environment. The diagram explains the integration between different network components such as OpenFlow switches, the Ryu Controller, Snort IDS, and the Game Theory-based decision engine.

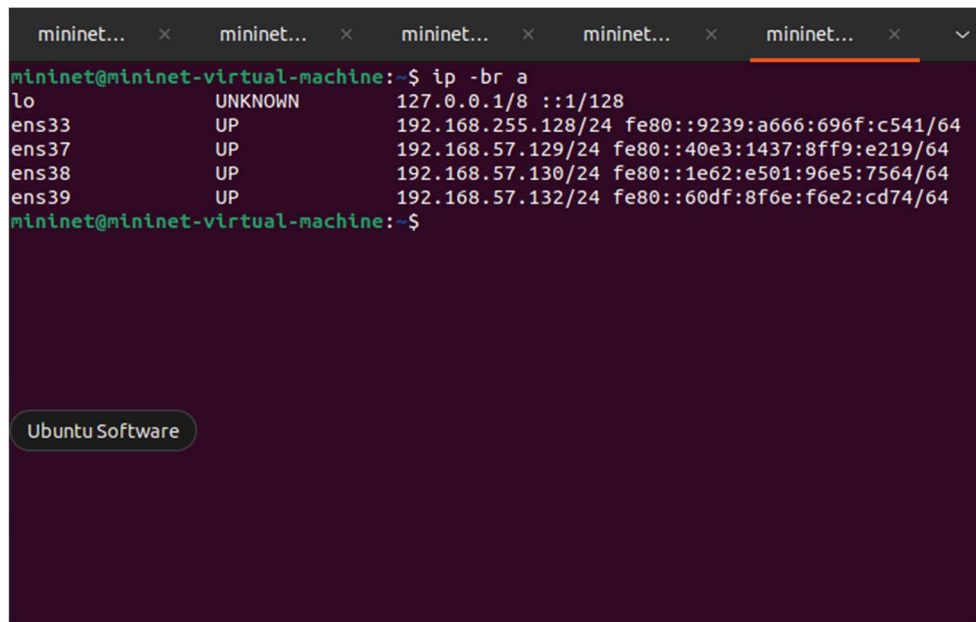
The topology consists of two main switches, Switch 1 and Switch 2, where multiple hosts are connected to each switch. In the figure, host h1 represents the attacker, while host h4 represents the victim. The topology is prepared to support later ICMP Flood and IP Spoofing experiments, where h1 can act as an attacker host and h4 can act as the victim host during the evaluation stage.

As traffic flows through the switches, a mirrored copy of the network traffic is sent to the Snort IDS using the Traffic Mirroring mechanism for real-time packet inspection and monitoring. Once suspicious activity is detected, Snort generates security alerts that are forwarded to the Alert Agent, which then sends them to the Ryu Controller.

Inside the Ryu Controller, the Game Theory Engine analyzes the network condition and selects the appropriate mitigation strategy based on the attack severity. The possible decisions include allowing normal traffic (ALLOW), applying different rate-limiting levels such as RL_1, RL_2, and RL_3, or applying BLOCK when the traffic becomes severe or persistent.

After selecting the mitigation strategy, the controller installs OpenFlow rules or Meter Rules into the switches to enforce the required security action directly within the network. The figure also demonstrates that monitoring, detection, and mitigation operate dynamically together to provide adaptive protection against network attacks.

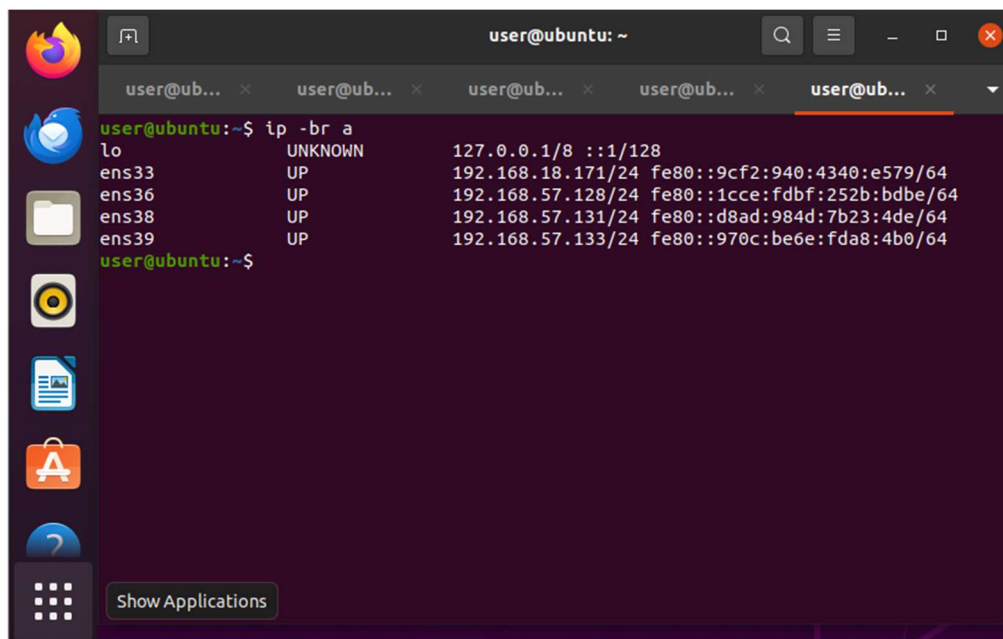
3.5 Network Interfaces and VM Layout



A terminal window titled 'mininet...' showing the output of the command 'ip -br a'. The output lists network interfaces: 'lo' (UNKNOWN, 127.0.0.1/8), 'ens33' (UP, 192.168.255.128/24), 'ens37' (UP, 192.168.57.129/24), 'ens38' (UP, 192.168.57.130/24), and 'ens39' (UP, 192.168.57.132/24). Each interface is also associated with a MAC address. The prompt is 'mininet@mininet-virtual-machine:~\$'.

```
mininet@mininet-virtual-machine:~$ ip -br a
lo                UNKNOWN      127.0.0.1/8  ::1/128
ens33             UP          192.168.255.128/24  fe80::9239:a666:696f:c541/64
ens37             UP          192.168.57.129/24  fe80::40e3:1437:8ff9:e219/64
ens38             UP          192.168.57.130/24  fe80::1e62:e501:96e5:7564/64
ens39             UP          192.168.57.132/24  fe80::60df:8f6e:f6e2:cd74/64
mininet@mininet-virtual-machine:~$
```

Figure 3.3 Mininet VM Network Interfaces.



A terminal window titled 'user@ubuntu: ~' showing the output of the command 'ip -br a'. The output lists network interfaces: 'lo' (UNKNOWN, 127.0.0.1/8), 'ens33' (UP, 192.168.18.171/24), 'ens36' (UP, 192.168.57.128/24), 'ens38' (UP, 192.168.57.131/24), and 'ens39' (UP, 192.168.57.133/24). Each interface is also associated with a MAC address. The prompt is 'user@ubuntu:~\$'.

```
user@ubuntu:~$ ip -br a
lo                UNKNOWN      127.0.0.1/8  ::1/128
ens33             UP          192.168.18.171/24  fe80::9cf2:940:4340:e579/64
ens36             UP          192.168.57.128/24  fe80::1cce:fdbf:252b:bdbe/64
ens38             UP          192.168.57.131/24  fe80::d8ad:984d:7b23:4de/64
ens39             UP          192.168.57.133/24  fe80::970c:be6e:fda8:4b0/64
user@ubuntu:~$
```

Figure 3.4 Snort VM Network Interfaces.

Table 3.4 VM Interfaces Used for Management and Monitoring.

VM	Interface	Purpose
Mininet VM	ens33	Management / Internet access
Mininet VM	ens37	Mirror output port for s1
Mininet VM	ens39	Mirror output port for s2
Snort VM	ens33	Management / SSH access
Snort VM	ens36	Monitoring interface for mirrored traffic
Snort VM	ens38	Monitoring interface for mirrored traffic

3.6 Mininet Network Topology Verification

After preparing the virtual machine interfaces, the Mininet topology was started to verify the SDN working environment. The topology consists of two OpenFlow switches, s1 and s2, connected to a centralized Ryu controller. Six hosts are deployed with IP addresses ranging from 10.0.0.1 to 10.0.0.6. Hosts h1, h2, and h3 are connected to s1, while hosts h4, h5, and h6 are connected to s2. In the later experimental stage, h1 is used as the attacker host and h4 is used as the main victim.

```
mininet@mininet-virtual-machine: ~  
Connecting to remote controller at 127.0.0.1:6653  
*** Adding hosts:  
h1 h2 h3 h4 h5 h6  
*** Adding switches:  
s1 s2  
*** Adding links:  
(h1, s1) (h2, s1) (h3, s1) (h4, s2) (h5, s2) (h6, s2) (s1, s2)  
*** Configuring hosts  
h1 h2 h3 h4 h5 h6  
*** Starting controller  
c0  
*** Starting 2 switches  
s1 s2 ...  
*** Starting CLI:  
mininet> nodes  
available nodes are:  
c0 h1 h2 h3 h4 h5 h6 s1 s2  
mininet> net  
h1 h1-eth0:s1-eth1  
h2 h2-eth0:s1-eth2  
h3 h3-eth0:s1-eth3  
h4 h4-eth0:s2-eth1  
h5 h5-eth0:s2-eth2  
h6 h6-eth0:s2-eth3  
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0 s1-eth4:s2-eth4  
s2 lo: s2-eth1:h4-eth0 s2-eth2:h5-eth0 s2-eth3:h6-eth0 s2-eth4:s1-eth4  
Ubuntu 22.04.5 LTS amd64  
*** Ping: testing ping reachability  
h1 -> h2 h3 h4 h5 h6  
h2 -> h1 h3 h4 h5 h6  
h3 -> h1 h2 h4 h5 h6  
h4 -> h1 h2 h3 h5 h6  
h5 -> h1 h2 h3 h4 h6  
h6 -> h1 h2 h3 h4 h5  
*** Results: 0% dropped (30/30 received)  
mininet>
```

Figure 3.5 Mininet Two-Switch Topology Verification.

Figure 3.5 verifies that the Mininet topology was created successfully. The output shows the available hosts, switches, controller, host-to-switch links, and inter-switch connection. The successful pingall result confirms that the baseline connectivity was working before executing any attack scenario.

3.7 Ryu Controller Initialization

The Ryu controller was started before launching the Mininet topology in order to manage the OpenFlow switches. This step verifies that the SDN control plane is active and ready to install forwarding or mitigation rules during later experiments.

```
mininet@mininet-virtual-machine: ~/SDN-GameTheory-Security/controllers  
mininet@mininet-virtual-machine:~/SDN-GameTheory-Security/controllers$ python3 -m py_compile ryu_d  
dos_controller.py  
python3 -m py_compile game_engine.py  
mininet@mininet-virtual-machine:~/SDN-GameTheory-Security/controllers$ ryu-manager ryu_ddos_contro  
ller.py  
1 RLock(s) were not greened, to fix this error make sure you run eventlet.monkey_patch() before im  
porting any other modules.  
loading app ryu_ddos_controller.py  
loading app ryu.controller.ofp_handler  
instantiating app ryu_ddos_controller.py of RyuDDoSController  
instantiating app ryu.controller.ofp_handler of OFPHandler
```

Figure 3.6 Ryu Controller Startup and OpenFlow Switch Connection.

Figure 3.6 shows that the Ryu controller started successfully and accepted connections from the OpenFlow switches. This confirms that the controller was ready to manage network flows in the emulated SDN environment.

3.8 Snort IDS Initialization

After preparing the monitoring interfaces, Snort IDS was initialized on the Snort VM. Snort is responsible for inspecting the mirrored traffic received from the Open vSwitch bridges. At this stage, Snort is only prepared for monitoring, while attack detection results are presented later in Chapter Five.

A terminal window with a dark purple background. The prompt is 'user@ubuntu:~\$'. The command 'snort -V' has been entered. The output shows the Snort++ logo, version 3.1.5.0, and a list of dependencies including DAQ, LuaJIT, OpenSSL, libpcap, PCRE, ZLIB, FlatBuffers, and LZMA.

```
user@ubuntu:~$ snort -V

  ,,-~
o" )~
  ' ' '

d.

-*> Snort++ <*-
Version 3.1.5.0
By Martin Roesch & The Snort Team
http://snort.org/contact#team
Copyright (C) 2014-2021 Cisco and/or its affiliates. All rights reserved.

Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using DAQ version 3.0.3
Using LuaJIT version 2.1.0-beta3
Using OpenSSL 1.1.1f 31 Mar 2020
Using libpcap version 1.9.1 (with TPACKET_V3)
Using PCRE version 8.44 2020-02-12
Using ZLIB version 1.2.11
Using FlatBuffers 1.12.0
Using LZMA version 5.2.4

user@ubuntu:~$
```

Figure 3.7 Snort IDS Initialization on the Monitoring Interface.

Figure 3.7 confirms that Snort IDS was installed and initialized successfully on the monitoring interface. This step prepares the detection layer to inspect mirrored traffic during the experimental phase.

3.9 Open vSwitch Traffic Mirroring Configuration

Open vSwitch was configured to support traffic mirroring from the SDN switches to the Snort VM. The monitoring interfaces were attached to the Open vSwitch bridges so that copied traffic could be forwarded to Snort without affecting the normal forwarding path.

```
mininet@mininet-virtual-machine: ~  
mininet@mininet-virtual-machine:~$ sudo ip link set ens37 up  
sudo ip addr flush dev ens37  
  
sudo ovs-vsctl --if-exists del-port s1 ens37  
sudo ovs-vsctl add-port s1 ens37  
  
sudo ovs-vsctl clear bridge s1 mirrors  
sudo ovs-vsctl -- --id=@p get port ens37 -- --id=@m create mirror name=m_s1_snort select-all=true  
output-port=@p -- set bridge s1 mirrors=@m  
[sudo] password for mininet:  
1750d189-9e43-45e3-9bf1-6fd7ef8ac6aa  
mininet@mininet-virtual-machine:~$ sudo ip link set ens39 up  
sudo ip addr flush dev ens39  
  
sudo ovs-vsctl --if-exists del-port s2 ens39  
sudo ovs-vsctl add-port s2 ens39  
  
sudo ovs-vsctl clear bridge s2 mirrors  
sudo ovs-vsctl -- --id=@p get port ens39 -- --id=@m create mirror name=m_s2_snort select-all=true  
output-port=@p -- set bridge s2 mirrors=@m  
220db5b7-a22b-46ab-b94f-42500682a010  
mininet@mininet-virtual-machine:~$ sudo ovs-vsctl list-ports s1  
sudo ovs-vsctl list-ports s2  
ens37  
s1-eth1  
s1-eth2  
s1-eth3  
s1-eth4  
ens39  
s2-eth1  
s2-eth2  
s2-eth3  
s2-eth4  
mininet@mininet-virtual-machine:~$
```

Figure 3.8 Open vSwitch Ports Connected to Monitoring Interfaces.

Figure 3.8 shows the Open vSwitch bridge configuration and verifies that the monitoring interfaces were attached to the virtual switches. This confirms that the switches were prepared to send mirrored traffic toward the Snort monitoring interfaces.

```
mininet@mininet-virtual-machine: ~  
mininet@mininet-virtual-machine:~$ sudo ovs-vsctl list mirror  
_uuid : 1750d189-9e43-45e3-9bf1-6fd7ef8ac6aa  
external_ids : {}  
name : m_s1_snort  
output_port : 3b8dfe2d-119f-4f7d-91c2-7a63630855b8  
output_vlan : []  
select_all : true  
select_dst_port : []  
select_src_port : []  
select_vlan : []  
snaplen : []  
statistics : {tx_bytes=1194, tx_packets=16}  
  
_uuid : 220db5b7-a22b-46ab-b94f-42500682a010  
external_ids : {}  
name : m_s2_snort  
output_port : 2c0d4217-ee3a-4052-a306-af2fa04a6a8b  
output_vlan : []  
select_all : true  
select_dst_port : []  
select_src_port : []  
select_vlan : []  
snaplen : []  
statistics : {tx_bytes=1194, tx_packets=16}  
mininet@mininet-virtual-machine:~$
```

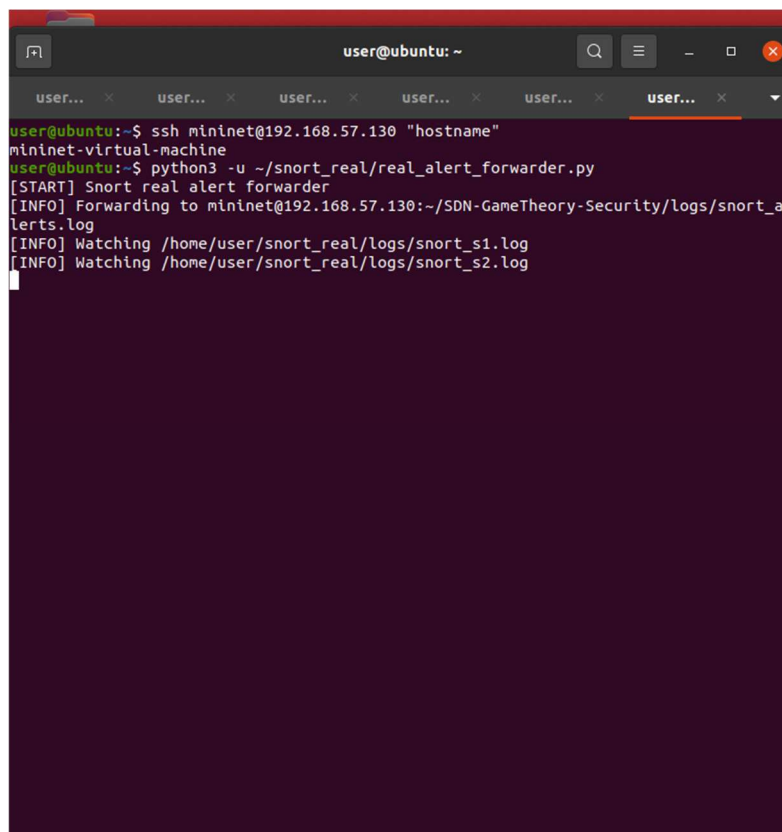
Figure 3.9 Open vSwitch Mirroring Configuration for s1 and s2.

Figure 3.9 verifies that traffic mirroring was configured successfully for both OpenFlow switches. The `select_all` option indicates that traffic passing through the selected switch is copied to the monitoring

output port. This configuration allows Snort to inspect network traffic externally without being placed directly inside the forwarding path.

3.10 Snort Alert Forwarding Agent Preparation

The Snort alert forwarding agent was prepared to connect the detection layer with the controller layer. The agent monitors Snort log files and forwards normalized alerts to the Mininet VM, where the Ryu controller can use them as part of the game-theoretic decision process.

A terminal window titled 'user@ubuntu: ~' showing the execution of an SSH command to connect to 'mininet' at IP '192.168.57.130'. After logging in as 'mininet-virtual-machine', the user runs a Python script 'real_alert_forwarder.py'. The script outputs a series of status messages: '[START] Snort real alert forwarder', '[INFO] Forwarding to mininet@192.168.57.130:~/SDN-GameTheory-Security/logs/snort_alerts.log', '[INFO] Watching /home/user/snort_real/logs/snort_s1.log', and '[INFO] Watching /home/user/snort_real/logs/snort_s2.log'.

```
user@ubuntu:~$ ssh mininet@192.168.57.130 "hostname"
mininet-virtual-machine
user@ubuntu:~$ python3 -u ~/snort_real/real_alert_forwarder.py
[START] Snort real alert forwarder
[INFO] Forwarding to mininet@192.168.57.130:~/SDN-GameTheory-Security/logs/snort_a
lerts.log
[INFO] Watching /home/user/snort_real/logs/snort_s1.log
[INFO] Watching /home/user/snort_real/logs/snort_s2.log
```

Figure 3.10 Snort Alert Forwarding Agent Startup.

Figure 3.10 shows that the alert forwarding agent started successfully and began monitoring the Snort log files. This verifies that the communication path between the Snort VM and the controller VM was prepared before running the attack experiments.

3.11 Chapter Summary

This chapter described the working environment used to implement the proposed SDN security framework. It presented the main tools and components used in the project, including Mininet, Open vSwitch, the Ryu Controller, Snort IDS, the alert forwarding agent, and the game-theoretic decision engine. The chapter also explained the virtual machine layout, the management and monitoring interfaces, and the role of traffic mirroring in forwarding copied traffic from the

OpenFlow switches to the Snort VM.

The Mininet topology was verified using two OpenFlow switches and six hosts, while the Ryu controller was initialized to manage the SDN control plane. Snort IDS was prepared on the monitoring interfaces to inspect mirrored traffic, and Open vSwitch mirroring was configured to support traffic visibility from both switches. Finally, the Snort alert forwarding agent was prepared to forward normalized alerts from the Snort VM to the controller VM.

Overall, this chapter confirmed that the experimental environment was correctly configured before executing the attack scenarios. The actual attack execution, Snort detection results, game-theoretic decisions, rate-limiting actions, blocking rules, and performance observations are presented in the following chapters.

4 Chapter Four: Proposed Solution

4.1 Introduction

This chapter presents the proposed adaptive security solution for detecting and mitigating DDoS-related threats in a Software-Defined Networking environment. The proposed framework combines traffic observation, Snort-based intrusion detection, alert forwarding, game-theoretic decision making, and OpenFlow-based mitigation enforcement. The main purpose of this chapter is to explain how the Ryu controller receives security-related observations, evaluates the current network state, estimates the severity of DDoS-related behavior, and selects the most suitable defensive action. The defensive actions include allowing normal traffic, applying graduated rate-limiting levels, or blocking malicious traffic when the observed behavior becomes severe, distributed, spoofed, or persistent. The proposed model is designed to avoid a one-response-for-all approach. Instead of treating all suspicious traffic in the same way, the controller evaluates multiple indicators such as packet rate, Snort alert status, packet loss, RTT, spoofing status, and attacker count. These indicators are then used by the game-theoretic decision model to compare the expected utility of each possible defense action. As a result, the selected mitigation action becomes proportional to the evaluated threat severity.

4.2 Overall Architecture

The proposed system is designed as a closed-loop SDN security framework. The workflow begins when network traffic is generated by Mininet hosts and forwarded through OpenFlow switches. In the experimental topology, normal hosts generate legitimate traffic, while attacker hosts may generate flooding or spoofed traffic toward a victim host. In the updated DDoS scenario, hosts h1, h2, and h3 act as distributed attackers, while h4 acts as the victim host. Open vSwitch forwards the original traffic according to the installed flow rules and simultaneously mirrors a copy of selected traffic to the Snort IDS virtual machine. Snort inspects the mirrored packets using predefined detection rules and generates alerts when suspicious ICMP flooding or spoofing-related behavior is detected. The Snort alert forwarding agent monitors the alert logs, extracts important information such as source IP address, destination IP address, protocol type, and alert message, and forwards a normalized alert to the Ryu controller. The controller then combines the received alert with additional observations collected from the SDN environment, such as packet rate, packet loss, RTT, spoofing status, and repeated suspicious rounds. After receiving and evaluating these observations, the game-theoretic decision engine estimates the current attacker behavior and calculates the expected utility of each possible defensive action. Based on this calculation, the controller selects one of the available mitigation strategies: ALLOW, RL_1, RL_2, RL_3, or BLOCK. The selected decision is then translated into OpenFlow meter rules or high-priority drop rules and enforced directly inside the Open vSwitch data plane.



Figure 4.1 Proposed Solution Workflow.

Figure 4.1 shows the logical workflow of the proposed solution. The framework begins by observing network traffic through Open vSwitch mirroring, then Snort IDS inspects the copied traffic and generates alerts when suspicious behavior is detected. The alert forwarding agent transfers normalized alerts to the controller side, where the Ryu controller combines these alerts with flow observations and passes them to the game-theoretic decision engine. The selected decision is finally enforced using OpenFlow meter rules or drop rules.

4.3 Observation and Threat Evaluation

The observation stage is responsible for collecting measurable indicators that describe the current traffic condition inside the SDN environment. Since the proposed framework aims to support adaptive mitigation, the decision engine does not depend on a single indicator. Instead, it combines traffic intensity, IDS-based evidence, and network performance indicators in order to evaluate whether the observed traffic is normal, suspicious, or flooding-related. The selected observations were chosen based on their practical availability in the implemented environment and their relevance to DDoS-related behavior. The Ryu Controller provides traffic-related information such as packet rate, while Snort IDS provides alert evidence when the mirrored traffic matches suspicious ICMP rules. In addition, connectivity tests provide performance-impact indicators such as packet loss and round-trip time (RTT). These indicators are commonly useful in SDN-based DDoS experiments because they reflect both the amount of generated traffic and its effect on network availability. In

the updated experimental scenario, the observation model also considers the number of suspicious traffic sources. This feature is important because the project is extended from a single-source flooding scenario into a multi-source emulated DDoS scenario, where h1, h2, and h3 generate ICMP flood traffic toward the victim host h4. Therefore, attacker_count is used to distinguish between a DoS-style flood from one host and a distributed flooding behavior generated by multiple hosts. The collected observations are used as inputs to the game-theoretic decision model. A high packet rate may indicate abnormal traffic intensity, but it is not sufficient alone to confirm an attack. Therefore, the decision engine evaluates it together with Snort alerts, packet loss, RTT, and the number of suspicious sources. This combination allows the controller to make a more reliable decision and select a suitable mitigation action such as ALLOW, RL_1, RL_2, RL_3, or BLOCK

Table 4.1 Threat Observation Interpretation Matrix.

Observed Condition	packet_rate_pps	snort_alert	packet_loss_pct	rtt_ms	attacker_count	Interpreted State	Explanation
Normal communication	Low	0	Low	Normal	1	NORMAL	Traffic does not show flooding indicators or IDS evidence.
Slight ICMP increase	Medium	0 or 1	Low	Slightly increased	1	PROBE / SUSPICIOUS	The traffic is not severe yet, but it requires monitoring or light rate limiting.
Single-source ICMP flood	High	1	Medium	Increased	1	DoS-oriented Flood	One source generates abnormal ICMP traffic toward the victim.
Multi-source ICMP flood	High	1	High	Increased	More than 1	DDoS-oriented Flood	Multiple suspicious sources target the same victim, indicating distributed flooding behavior.
Spoofed ICMP flood	High	1	Medium or High	Increased	1 or more	Spoofed DDoS-related Flood	The traffic combines flooding behavior with identity manipulation.
Persistent attack	High over repeated rounds	1	High	High	1 or more	Persistent DDoS	The suspicious behavior continues across repeated decision rounds.

Table 4.1 explains how the raw observations are interpreted by the proposed framework. A high packet rate alone is not sufficient to classify a DDoS-related attack because legitimate traffic may occasionally increase. Therefore, the framework combines packet rate with Snort alert status, packet loss, RTT, spoofing indicators, and attacker_count. The attacker_count feature is particularly

important because it separates single-source DoS behavior from multi-source DDoS-oriented behavior. When several suspicious sources target the same victim, the evaluated severity increases and the decision engine becomes more likely to select RL_3 or BLOCK.

4.4 Game Theory-Based Decision Model

The proposed framework models the security problem as a repeated attacker–defender game. The attacker represents the traffic source that may behave normally, generate probing traffic, launch flooding traffic, or use spoofed traffic. The defender is represented by the Ryu controller, which observes network behavior and selects the most suitable mitigation action. Figure 4.2 illustrates the decision model used by the proposed framework.

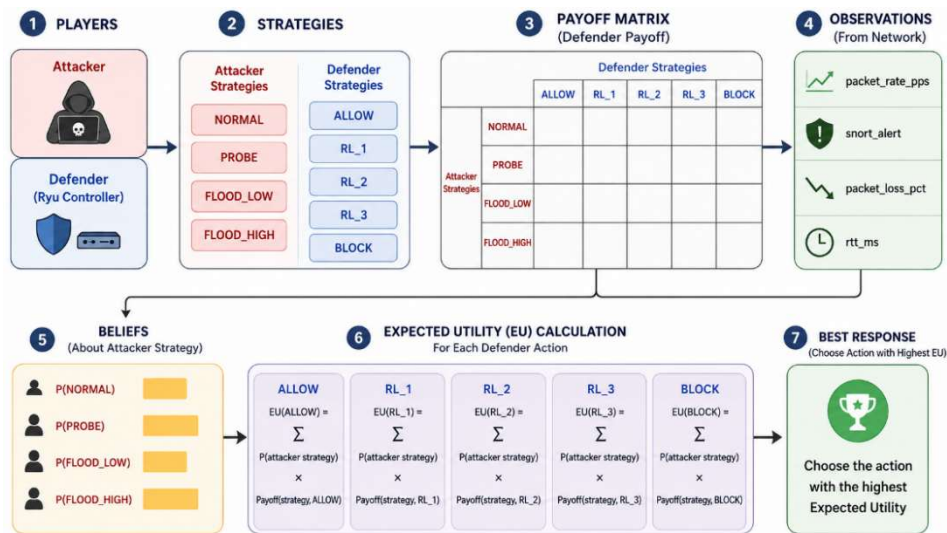


Figure 4.2 Game Theory Model for Optimal Defense Selection.

Figure 4.2 presents the overall game-theoretic decision framework used in the proposed SDN security system. The framework models the interaction between the attacker and the defender, defines the available strategies for both sides, uses observed network indicators to estimate attacker behavior, computes the expected utility of each defender action, and then selects the best response.

4.4.1 Players

The proposed model consists of two main players: the attacker and the defender. The attacker represents one or more hosts that generate suspicious or malicious traffic toward a victim host. In the DDoS-oriented scenario of this project, hosts h1, h2, and h3 are treated as distributed attack sources, while h4 represents the victim host. This reflects a multi-source flooding behavior rather than a single-source DoS attack.

The defender is represented by the Ryu controller supported by the game-theoretic decision engine. The defender observes the current network condition, receives Snort alerts, evaluates the severity of the traffic, and selects the most suitable mitigation strategy. Therefore, the security interaction is modeled as an attacker–defender game in which the attacker attempts to increase disruption, while the defender attempts to preserve network availability and reduce attack impact.

4.4.2 Attacker Strategy

The attacker strategy describes the possible behavior that may be observed in the network. In this project, the attacker is not limited to one fixed behavior. Instead, the model considers several possible states, ranging from normal communication to severe DDoS-related flooding. This is important because the controller should not immediately block traffic whenever the packet rate increases. Instead, it should evaluate the type and severity of the behavior.

Table 4.2 Attacker Strategy Space.

Attacker Strategy	Description	DDoS Relevance
NORMAL	Legitimate communication without flooding indicators	No attack
PROBE	Low or moderate suspicious traffic that may indicate scanning or preparation	Early warning
DoS Flood	High-rate ICMP traffic from one source toward the victim	Single-source flooding
DDoS Flood	High-rate ICMP traffic from multiple sources toward the same victim	Main DDoS scenario
Spoofed DDoS	Flooding traffic combined with spoofed or manipulated source identity	Severe attack case

The distinction between DoS Flood and DDoS Flood is important in this project. A DoS flood is generated by one suspicious source, while a DDoS flood is represented by several suspicious sources targeting the same victim. Therefore, the `attacker_count` feature is used to support this distinction.

4.4.3 Defender Strategy

The defender strategy represents the possible actions that the Ryu controller can apply after evaluating the current network condition. The proposed framework uses graduated mitigation rather than one fixed response. This allows the controller to avoid unnecessary blocking during normal or slightly suspicious traffic, while still applying stronger actions when the attack becomes severe, distributed, spoofed, or persistent.

Table 4.3 Defender Strategy Space.

Defender Strategy	Action Type	Purpose
ALLOW	Normal forwarding	Preserve normal communication
RL_1	Light rate limiting	Reduce low suspicious traffic
RL_2	Medium rate limiting	Control confirmed suspicious traffic
RL_3	Strong rate limiting	Reduce severe or distributed flooding
BLOCK	Drop rule	Stop severe, spoofed, or persistent malicious traffic

This strategy space supports adaptive mitigation because the controller can move gradually from ALLOW to RL_1, RL_2, RL_3, and finally BLOCK depending on the evaluated attack severity.

4.4.4 Observation Vector

The observation vector represents the set of measurable indicators used by the controller to evaluate the current traffic condition. In this project, the observation vector combines traffic intensity, IDS evidence, network performance impact, and the number of suspicious sources.

$$O = \{R, A, L, D, C, S\}$$

Where:

R: Normalized packet rate.

A: Snort alert status.

L: Packet loss ratio.

D: RTT delay factor.

C: Attacker count factor.

S: Spoofing indicator.

In this model, packet rate alone is not sufficient to classify a DDoS-related attack. Legitimate traffic may also increase temporarily. Therefore, the controller evaluates packet rate together with Snort alerts, packet loss, RTT, spoofing status, and attacker count. The attacker_count feature is especially important because it helps distinguish a single-source DoS-style flood from a multi-source DDoS-oriented flood.

4.4.5 Threat Score Calculation

The threat score is used to summarize the severity of the current observation. It is not the final decision by itself. Instead, it provides an intermediate value that helps the decision engine estimate attacker behavior and compare possible mitigation actions.

$$T = w1R + w2A + w3L + w4D + w5C + w6S$$

where:

- T denotes the overall threat score.
- R represents the normalized packet rate.
- A denotes the Snort alert status.
- L represents the packet loss ratio.
- D denotes the RTT delay factor.
- C represents the attacker count factor.
- S denotes the spoofing indicator.
- $w1, \dots, w6$
are weighting coefficients assigned to the corresponding features.

The values $w1$ to $w6$ are dimensionless weight values. They are used to reflect the importance of each feature in the DDoS evaluation process. For example, Snort alerts and attacker_count are given higher importance in DDoS-oriented cases because IDS evidence and multiple suspicious sources provide stronger indicators than packet rate alone. The threat score is dynamic because it changes whenever the observed traffic values change.

4.4.6 Belief Estimation

Belief estimation converts the observed indicators into a probability distribution over the possible attacker strategies. This step is important because the controller does not assume one fixed attack state. Instead, it estimates how likely each attacker strategy is under the current observation.

$$P(a_j|O) = \frac{\text{Score}(a_j)}{\sum_{k=1}^n \text{Score}(a_k)}$$

Where:

- $P(a_j | O)$: Probability of attacker strategy a_j given the observation vector O .
- $\text{Score}(a_j)$: Raw score assigned to attacker strategy a_j .
- n : Number of attacker strategies.
- O : Observation vector.

The probabilities must sum to one. This allows the defender to compare the expected utility of each mitigation action. For example, if the observed packet rate is high, Snort alert is active, packet loss increases, and attacker_count is greater than one, the belief assigned to the DDoS Flood state becomes higher than the belief assigned to NORMAL traffic.

4.4.7 Defender Payoff Matrix

The defender payoff matrix shows how suitable each defense action is against each possible attacker strategy. The values in the matrix are utility values, not percentages and not packet rates. A higher value means that the action is more suitable for that attack state.

Table 4.4 Defender Payoff Matrix.

Defender Action / Attacker State	NORMAL	PROBE	DoS Flood	DDoS Flood	Spoofed DDoS
ALLOW	10	4	-6	-8	-10
RL ₁	6	7	2	0	-2
RL ₂	2	5	7	5	2
RL ₃	-1	2	6	8	6
BLOCK	-8	-3	3	7	10

The payoff values are selected according to the expected benefit of each defense action. For example, ALLOW has the highest value under NORMAL traffic because it preserves service availability. However, ALLOW has negative values under DDoS-related states because allowing severe flooding may damage network performance. BLOCK has a negative value under NORMAL traffic because blocking legitimate traffic causes unnecessary service disruption, but it has a high value under Spoofed DDoS because spoofed flooding represents a severe threat.

4.4.8 Expected Utility Calculation

Expected utility is used to compare all defender actions under the current estimated attacker behavior. The controller calculates the expected utility of each possible defense action and then selects the action with the highest value.

$$EU(d_i) = \sum_{\{j=1\}}^{\{n\}P(a_j|O) \times} M(d_i, a_j) - C(d_i)$$

Where:

- $EU(d_i)$: Expected utility of defender action.
- $P(a_j | O)$: Probability of attack a_j given the observation O .
- $M(d_i, a_j)$: Payoff value from the defender payoff matrix.
- $C(d_i)$: Operational cost of applying the defense action.
- d_i : Defender action such as ALLOW, RL₁, RL₂, RL₃, or BLOCK.
- a_j : Possible attack action.
- n : Total number of possible attack actions.

The cost term $C(di)$ represents the operational impact of each defense action. For example, ALLOW has a low cost because it does not restrict traffic, while BLOCK has a higher cost because it may interrupt communication if applied incorrectly. Therefore, the selected action is not only the strongest action, but the action that gives the best balance between protection and service preservation.

Example of DDoS-Oriented Utility Comparison

Assume that the belief estimation process produces the following probability distribution for attacker states:

$$\begin{aligned} P_{\text{NORMAL}} &= 0.05 \\ P_{\text{PROBE}} &= 0.10 \\ P_{\text{DoS}} &= 0.20 \\ P_{\text{DDoS}} &= 0.55 \\ P_{\text{Spoofed}} &= 0.10 \end{aligned}$$

In this case, DDoS Flood has the highest probability. The controller then compares the expected utility of ALLOW, RL_1, RL_2, RL_3, and BLOCK. Since the DDoS probability is high, RL_3 or BLOCK becomes more suitable than ALLOW. However, if the traffic is not spoofed and not persistent enough, RL_3 may be selected before BLOCK in order to reduce the attack impact while still preserving some legitimate communication.

4.4.9 Optimal Defense Selection

The final mitigation action is selected based on the highest expected utility value. This means that the controller does not select a fixed response for all attacks. Instead, it evaluates the current observation, estimates attacker behavior, calculates expected utility values, and then chooses the best action.

$$d^* = \arg\max_{d_i \in D} EU(d_i)$$

Where:

- d^* : Selected optimal defense action.
- D : Set of available defender actions.
- $EU(di)$: Expected utility of defense action di .

The selected action is then passed to the enforcement stage. If the selected action is ALLOW, the traffic is forwarded normally. If the selected action is RL_1, RL_2, or RL_3, the controller applies a rate-limiting rule. If the selected action is BLOCK, the controller installs a high-priority OpenFlow drop rule. This process makes the proposed defense adaptive, proportional, and suitable for DDoS-oriented SDN protection.

4.5 Snort Rules and Alert Forwarding Mechanism

Snort rules are used to detect suspicious traffic patterns in the mirrored traffic received from the OpenFlow switches. In this project, Snort alerts are used as IDS-based evidence for ICMP flooding and spoofing-related behavior. The purpose of Snort is not to apply mitigation directly, but to generate alerts that support the controller-side decision process.

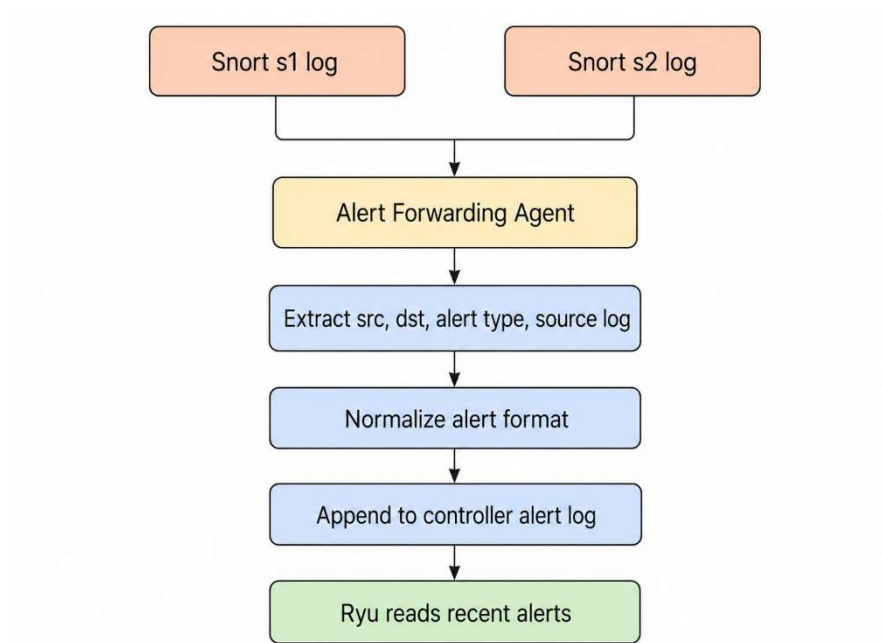


Figure 4.3 Snort Alert Forwarding Mechanism.

Figure 4.3 shows the logical design of the alert forwarding mechanism. The agent monitors the Snort log files generated for the mirrored traffic from both switches. When a relevant alert is found, the agent extracts the source IP address, destination IP address, alert type, and log source, then forwards a normalized alert entry to the controller-side alert log. This design separates detection from decision-making while allowing the controller to use IDS alerts in its mitigation process.

4.6 Mitigation Strategy Selection

The proposed framework uses graduated mitigation actions rather than a single fixed response. Table 4.2 summarizes the defensive actions supported by the game-theoretic decision engine.

Table 4.5 Mitigation Actions Used by the Proposed Framework.

Decision	Rate / Action	Meaning
ALLOW	10000 kbps	Normal traffic is allowed without restriction

Decision	Rate / Action	Meaning
RL_1	4000 kbps	Light rate limiting for low suspicious behavior
RL_2	1024 kbps	Medium rate limiting for confirmed suspicious behavior
RL_3	512 kbps	Strong rate limiting for severe flooding behavior
BLOCK	Drop rule	Severe, spoofed, or persistent malicious traffic is blocked

This graduated strategy allows the controller to respond proportionally to the observed threat level. Low-risk traffic is allowed or lightly rate-limited, while severe or persistent attacks are blocked using an OpenFlow drop rule.

After receiving alerts from Snort IDS, the Ryu Controller analyzes the traffic state and selects a mitigation strategy. The system supports multiple defensive actions depending on attack severity. Normal traffic may be allowed, low-level suspicious traffic may be rate-limited, and severe attacks may be blocked using OpenFlow rules.

4.7 OpenFlow-Based Enforcement

After the game-theoretic decision engine selects a mitigation action, the Ryu controller translates the decision into OpenFlow behavior inside the Open vSwitch data plane. Table 4.3 shows how each decision is enforced.

Table 4.6 OpenFlow Enforcement Mapping.

Controller Decision	OpenFlow Enforcement Method	Expected Effect
ALLOW	Normal forwarding rule	Traffic continues normally
RL_1	Meter rule with 4000 kbps	Light reduction of suspicious traffic
RL_2	Meter rule with 1024 kbps	Moderate traffic restriction
RL_3	Meter rule with 512 kbps	Strong traffic restriction
BLOCK	High-priority drop rule	Malicious traffic is dropped

Rate-limiting decisions are enforced using OpenFlow meter-based rules, while the BLOCK decision is enforced using a high-priority drop rule. This makes the controller decision operational inside the forwarding layer.

In the proposed framework, rate-limiting decisions are enforced using OpenFlow meter rules, while the BLOCK decision is enforced using a high-priority drop rule. This distinction allows the

system to reduce suspicious traffic gradually when the threat is moderate, and to completely stop the flow when the threat becomes severe, spoofed, or persistent.

4.8 Chapter Summary

This chapter presented the proposed DDoS-oriented SDN security solution. It explained the overall architecture of the framework, including Mininet traffic generation, Open vSwitch forwarding and mirroring, Snort-based detection, alert forwarding, game-theoretic decision making, and OpenFlow-based enforcement.

The chapter also clarified how raw observations are interpreted before making a mitigation decision. These observations include packet rate, Snort alert status, packet loss, RTT, spoofing indicators, and attacker count. The `attacker_count` feature was added to distinguish between single-source DoS behavior and multi-source DDoS-oriented flooding behavior.

In addition, the chapter explained the attacker and defender strategies, the observation vector, threat score calculation, belief estimation, payoff matrix, expected utility calculation, and optimal defense selection. This structure shows that the proposed framework does not rely on a fixed threshold only. Instead, it compares possible defensive actions and selects the most suitable response according to the evaluated attack severity.

Finally, the chapter described how Snort alerts are forwarded to the controller and how the selected mitigation decision is translated into OpenFlow rules. Rate-limiting decisions are enforced using meter rules, while severe, spoofed, distributed, or persistent attacks are handled using high-priority drop rules

5 Chapter Five: Experimental Results and Evaluation

5.1 Introduction

This chapter presents the experimental results of the proposed SDN security framework. The evaluation focuses on verifying the complete workflow of the system, starting from traffic generation in Mininet, traffic mirroring through Open vSwitch, detection by Snort IDS, alert forwarding to the Ryu controller, game-theoretic decision making, and final mitigation using OpenFlow rules. The experiments include normal traffic, ICMP flood traffic, and spoofed ICMP flood traffic. The results show how the controller changes its response from ALLOW to different rate-limiting levels and finally to BLOCK when the attack becomes severe or persistent.

5.2 Summary of Experimental Results

Table 5.1 Summary of Experimental Results

Scenario	Observation	Controller Decision	Enforcement Result
Normal Traffic	Low packet rate and no Snort alert	ALLOW	Traffic forwarded normally
Mild Suspicious Traffic	Slight increase in ICMP activity	RL_1	Light rate limiting applied at 4000 kbps
Moderate ICMP Flood	Snort alert received and suspicious packet rate detected	RL_2	Medium rate limiting applied at 1024 kbps
Strong ICMP Flood	Higher flood probability and repeated suspicious behavior	RL_3	Strong rate limiting applied at 512 kbps
Persistent ICMP Flood	Attack continues across multiple rounds	BLOCK	High-priority OpenFlow drop rule installed
Spoofed ICMP Flood	Fake source IP detected, such as 10.0.0.99	BLOCK	Spoofed traffic dropped using OpenFlow rule

The results show that the proposed framework does not apply a fixed response to all traffic. Instead, the Ryu controller evaluates traffic behavior in repeated decision rounds. As the attack indicators become stronger or persist for a longer period, the selected mitigation action becomes stricter. This confirms the adaptive behavior of the game-theoretic decision engine.

5.3 Game-Theoretic Decision Progression

The controller selected different mitigation actions depending on the observed traffic state. During normal traffic, the controller selected ALLOW because the flow did not show attack indicators. When suspicious traffic started to appear, RL_1 was selected as a light mitigation action. With stronger ICMP activity and Snort alerts, the decision escalated to RL_2. In more severe rounds, RL_3 was selected to apply stricter rate limiting. Finally, when the attack continued across multiple rounds or when spoofed traffic was detected, the controller selected BLOCK and installed a high-priority OpenFlow drop rule.

5.4 Snort-Based ICMP Flood Detection

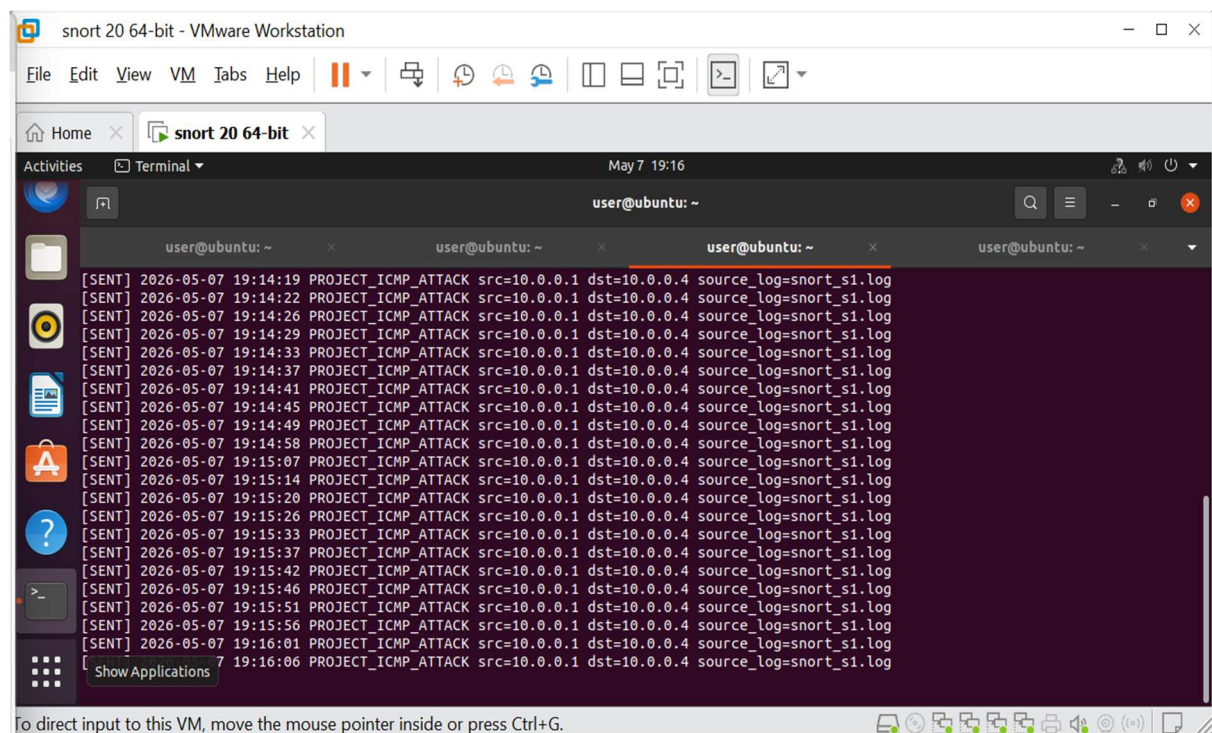


Figure 5.1 ICMP Attack Detection Using Snort IDS in the SDN Environment.

This figure demonstrates the detection of network attacks using the Snort Intrusion Detection System (IDS) within the Software Defined Networking (SDN) environment.

The displayed logs show that the system successfully detected multiple ICMP-based attacks while continuously monitoring network traffic.

The logs include detailed information about each detected event, such as:

Attack detection timestamp

Detected attack type (PROJECT_ICMP_ATTACK)

Source IP address

Destination IP address

The log file where the alert was stored

The results indicate that the source address:

10.0.0.1

was sending a large number of ICMP packets toward the target device:

10.0.0.4

which suggests suspicious behavior or a potential ICMP flooding attack.

The system relies on custom Snort rules to analyze network packets and compare traffic behavior against predefined detection rules. Whenever abnormal activity is identified, the system generates and records an alert.

The figure also shows that alerts are stored inside the file:

snort_s1.log

to maintain a complete record of detected attacks for further analysis and investigation.

These results confirm the successful:

Deployment of Snort IDS

Continuous monitoring of network traffic

Real-time detection of ICMP attacks

Logging and analysis of secu

rity events within the SDN environment.

5.5 Adaptive Defense and Rate-Limiting Decisions

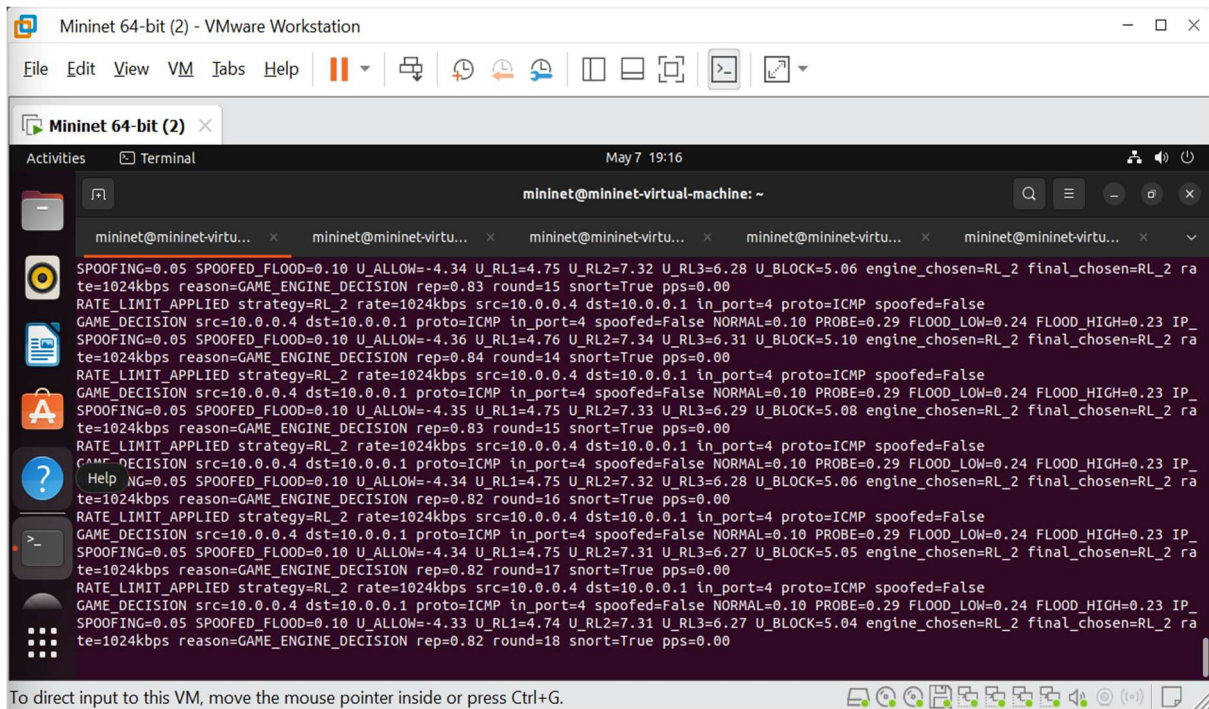


Figure 5.2 Dynamic Defense Mechanisms and Suspicious Traffic Rate Limiting in the SDN Environment.

This figure demonstrates the implementation of dynamic defensive decision-making within the Software Defined Networking (SDN) environment, where the system continuously analyzes network traffic and selects the appropriate security response based on the detected threat level.

The displayed logs contain detailed traffic-related information, including:

Detected attack type

Packet transmission rate

Protocol type

Port numbers

Traffic analysis results

Final security decisions generated by the system

The results show the detection of multiple suspicious activities, including:

Flooding attacks

Spoofing attacks

Network probing activities

After evaluating the threat severity, the Game-Theory-based decision engine dynamically selects the most appropriate defensive action.

Several security actions are visible in the logs, such as:

ALLOW for legitimate traffic

RL_1 for low-level rate limiting

RL_2 for medium-level rate limiting

RL_3 for high-level rate limiting

BLOCK for completely blocking malicious traffic

The figure also demonstrates the application of:

RATE_LIMIT_APPLIED

which represents the execution of traffic control mechanisms to mitigate attacks and reduce network resource exhaustion.

These results confirm the ability of the proposed system to:

Analyze threats in real time

Differentiate between normal and malicious traffic

Apply flexible defensive strategies

Reduce the impact of attacks on the network

Enhance security within the SDN environment

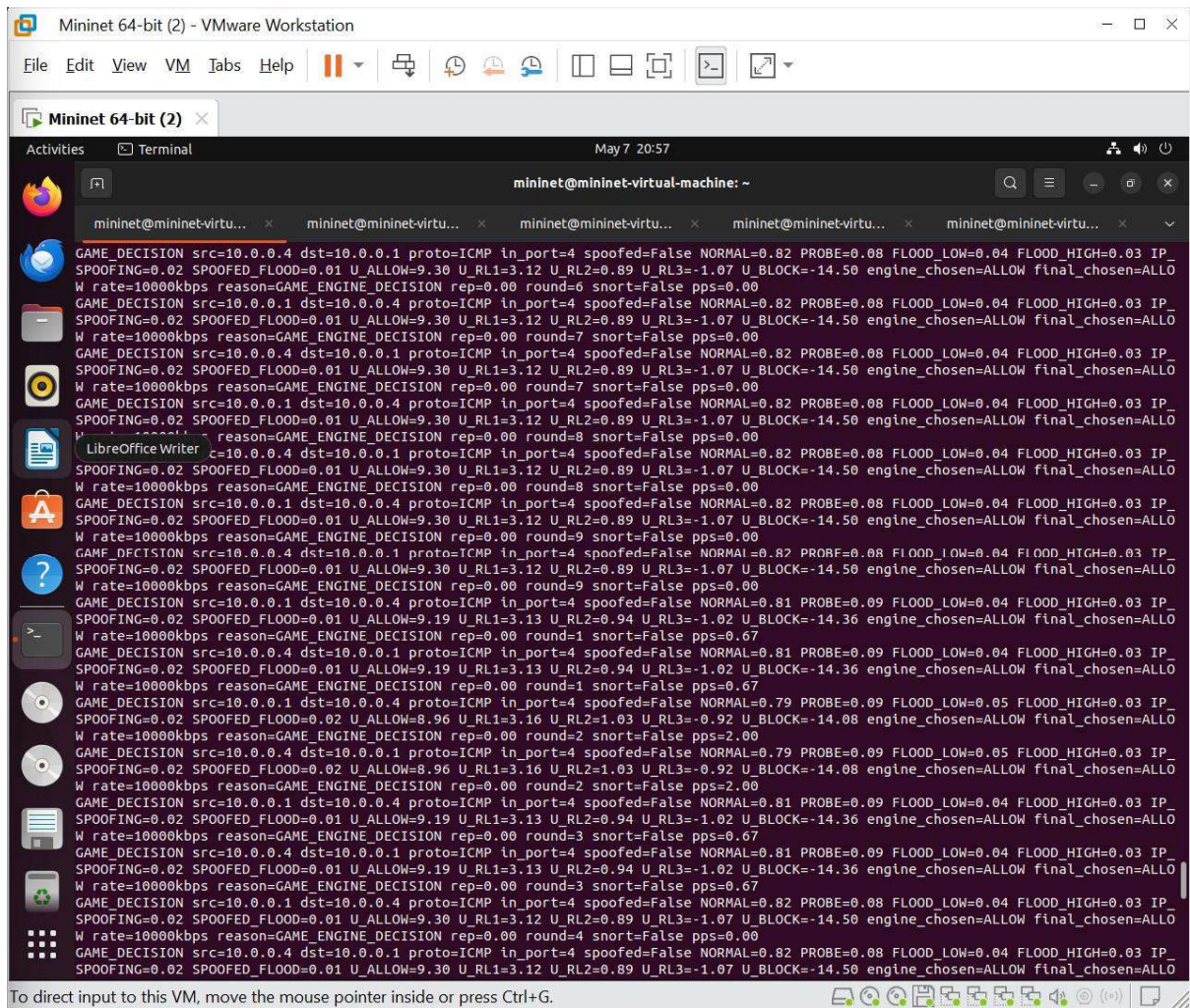


Figure 5.3 Real-Time Attack Detection and Threat Assessment Logs in the SDN Environment.

This figure illustrates the real-time monitoring and analysis process within the Software Defined Networking (SDN) environment, where the system continuously displays traffic analysis logs and security decisions during network operation.

The logs demonstrate the system's ability to classify network behavior into multiple categories, including:

NORMAL traffic

Network probing activities (PROBE)

Spoofing attacks (SPOOF)

Flooding attacks (FLOOD)

The displayed logs also contain detailed information related to each detected event, such as:

Source and destination addresses

Protocol type

Port numbers

Packet transmission rate

Threat level or detected attack category

The implemented environment integrates:

Ryu Controller

Mininet

Snort IDS

A Game-Theory-based decision mechanism to analyze threats and generate dynamic defensive responses in real time.

The figure additionally shows how the system applies different security actions, including:

ALLOW for legitimate traffic

RL_1, RL_2, and RL_3 for applying different traffic rate-limiting policies against suspicious activities

These results demonstrate the capability of the proposed system to:

Continuously monitor network traffic

Detect attacks in real time

Assess threat severity

Apply flexible and dynamic defensive actions within the SDN environment

Figure 5.4 Dynamic Defense Decision Execution and Mitigation Strategy Enforcement in the SDN Environment.

Figure (5.4) illustrates the practical implementation results of the dynamic decision-making engine within the Software-Defined Networking (SDN) environment during an ICMP Flood attack simulation.

The logs demonstrate how the system analyzes network traffic behavior based on several security indicators, including attack probability, spoofing probability, packets per second (PPS), and Snort IDS alert status.

Based on these observations, the Game Theory Engine calculates the utility values of multiple defense strategies, including:

ALLOW, RL_1, RL_2, RL_3, and BLOCK.

The results show that the framework does not rely on static threshold-based decisions. Instead, it dynamically selects the most appropriate mitigation strategy according to the attack severity and attacker behavior in real time.

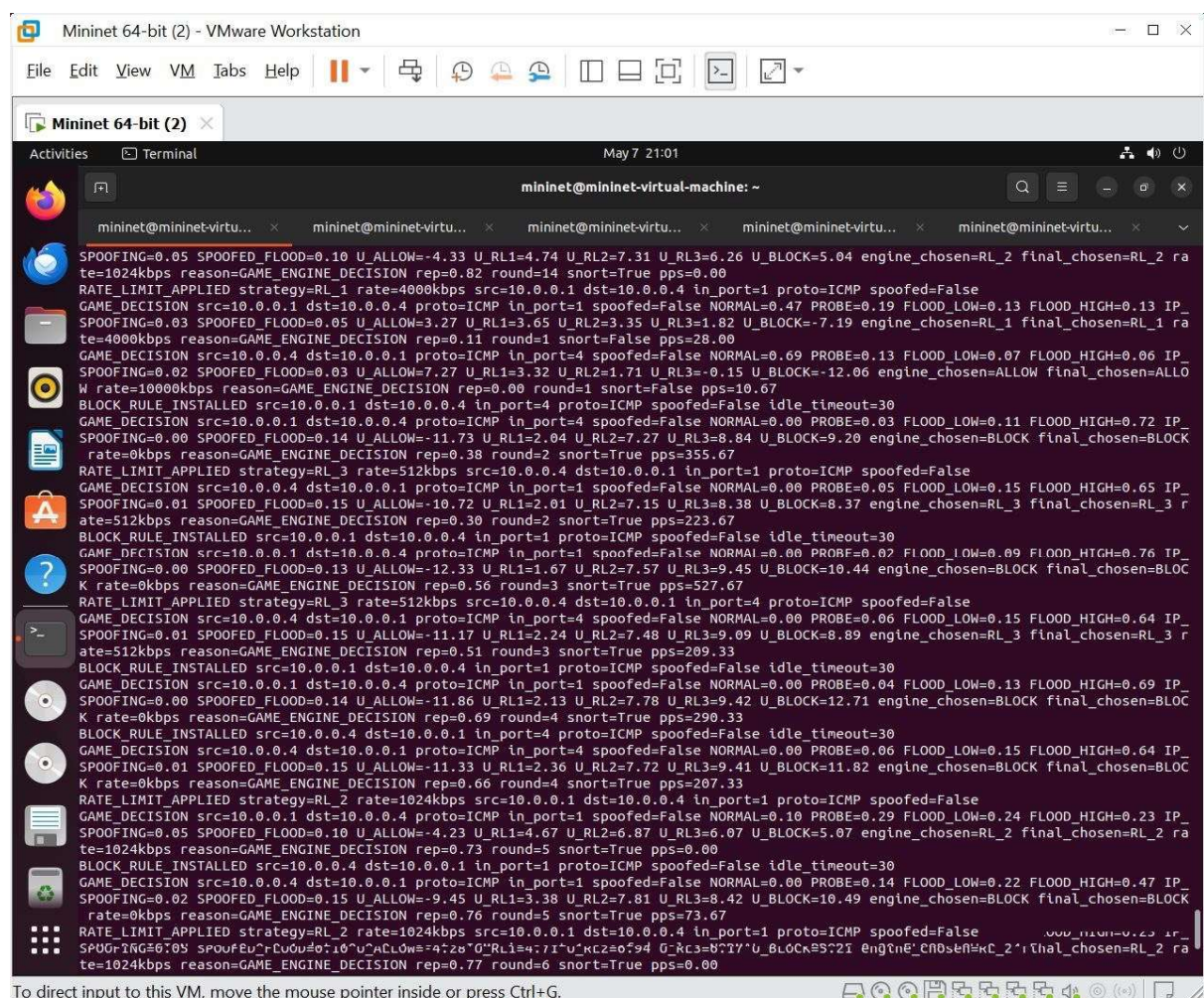
The figure also demonstrates several defensive actions enforced by the Ryu Controller, including:

Applying rate-limiting policies

Installing blocking rules (BLOCK_RULE_INSTALLED)

Reducing malicious traffic rates

These actions contribute to protecting the SDN environment against flooding attacks and improving network resilience.



```
mininet@mininet-virtual-machine: ~  
mininet@mininet-virtu... x mininet@mininet-virtu... x mininet@mininet-virtu... x mininet@mininet-virtu... x mininet@mininet-virtu... x  
SPOOFING=0.05 SPOOFED_FLOOD=0.10 U_ALLOW=-4.33 U_RL1=4.74 U_RL2=7.31 U_RL3=6.26 U_BLOCK=5.04 engine_chosen=RL_2 final_chosen=RL_2 ra  
te=1024kbps reason=GAME_ENGINE_DECISION rep=0.82 round=14 snort=True pps=0.00  
RATE LIMIT APPLIED strategy=RL_1 rate=4000kbps src=10.0.0.1 dst=10.0.0.4 in_port=1 proto=ICMP spoofed=False  
GAME DECISION src=10.0.0.1 dst=10.0.0.4 proto=ICMP in_port=1 spoofed=False NORMAL=0.47 PROBE=0.19 FLOOD_LOW=0.13 FLOOD_HIGH=0.13 IP_  
SPOOFING=0.03 SPOOFED_FLOOD=0.05 U_ALLOW=3.27 U_RL1=3.65 U_RL2=3.35 U_RL3=1.82 U_BLOCK=-7.19 engine_chosen=RL_1 final_chosen=RL_1 ra  
te=4000kbps reason=GAME_ENGINE_DECISION rep=0.11 round=1 snort=False pps=28.00  
GAME DECISION src=10.0.0.4 dst=10.0.0.1 proto=ICMP in_port=4 spoofed=False NORMAL=0.69 PROBE=0.13 FLOOD_LOW=0.07 FLOOD_HIGH=0.06 IP_  
SPOOFING=0.02 SPOOFED_FLOOD=0.03 U_ALLOW=7.27 U_RL1=3.32 U_RL2=1.71 U_RL3=-0.15 U_BLOCK=-12.06 engine_chosen=ALLOW final_chosen=ALLO  
W rate=10000kbps reason=GAME_ENGINE_DECISION rep=0.00 round=1 snort=False pps=10.67  
BLOCK_RULE_INSTALLED src=10.0.0.1 dst=10.0.0.4 in_port=4 proto=ICMP spoofed=False idle timeout=30  
GAME DECISION src=10.0.0.1 dst=10.0.0.4 proto=ICMP in_port=4 spoofed=False NORMAL=0.00 PROBE=0.03 FLOOD_LOW=0.11 FLOOD_HIGH=0.72 IP_  
SPOOFING=0.00 SPOOFED_FLOOD=0.14 U_ALLOW=-11.73 U_RL1=2.04 U_RL2=7.27 U_RL3=8.84 U_BLOCK=9.20 engine_chosen=BLOCK final_chosen=BLOCK  
rate=0kbps reason=GAME_ENGINE_DECISION rep=0.38 round=2 snort=True pps=355.67  
RATE LIMIT APPLIED strategy=RL_3 rate=512kbps src=10.0.0.4 dst=10.0.0.1 in_port=1 proto=ICMP spoofed=False  
GAME DECISION src=10.0.0.4 dst=10.0.0.1 proto=ICMP in_port=1 spoofed=False NORMAL=0.00 PROBE=0.05 FLOOD_LOW=0.15 FLOOD_HIGH=0.65 IP_  
SPOOFING=0.01 SPOOFED_FLOOD=0.15 U_ALLOW=-10.72 U_RL1=2.01 U_RL2=7.15 U_RL3=8.38 U_BLOCK=8.37 engine_chosen=RL_3 final_chosen=RL_3 r  
ate=512kbps reason=GAME_ENGINE_DECISION rep=0.30 round=2 snort=True pps=223.67  
BLOCK_RULE_INSTALLED src=10.0.0.1 dst=10.0.0.4 in_port=1 proto=ICMP spoofed=False idle timeout=30  
GAME DECISION src=10.0.0.1 dst=10.0.0.4 proto=ICMP in_port=1 spoofed=False NORMAL=0.00 PROBE=0.02 FLOOD_LOW=0.09 FLOOD_HIGH=0.76 IP_  
SPOOFING=0.00 SPOOFED_FLOOD=0.13 U_ALLOW=-12.33 U_RL1=1.67 U_RL2=7.57 U_RL3=9.45 U_BLOCK=10.44 engine_chosen=BLOCK final_chosen=BL0C  
K rate=0kbps reason=GAME_ENGINE_DECISION rep=0.56 round=3 snort=True pps=527.67  
RATE LIMIT APPLIED strategy=RL_3 rate=512kbps src=10.0.0.4 dst=10.0.0.1 in_port=4 proto=ICMP spoofed=False  
GAME DECISION src=10.0.0.4 dst=10.0.0.1 proto=ICMP in_port=4 spoofed=False NORMAL=0.00 PROBE=0.06 FLOOD_LOW=0.15 FLOOD_HIGH=0.64 IP_  
SPOOFING=0.01 SPOOFED_FLOOD=0.15 U_ALLOW=-11.17 U_RL1=2.24 U_RL2=7.48 U_RL3=9.09 U_BLOCK=8.89 engine_chosen=RL_3 final_chosen=RL_3 r  
ate=512kbps reason=GAME_ENGINE_DECISION rep=0.51 round=3 snort=True pps=209.33  
BLOCK_RULE_INSTALLED src=10.0.0.1 dst=10.0.0.4 in_port=1 proto=ICMP spoofed=False idle timeout=30  
GAME DECISION src=10.0.0.1 dst=10.0.0.4 proto=ICMP in_port=1 spoofed=False NORMAL=0.00 PROBE=0.04 FLOOD_LOW=0.13 FLOOD_HIGH=0.69 IP_  
SPOOFING=0.00 SPOOFED_FLOOD=0.14 U_ALLOW=-11.86 U_RL1=2.13 U_RL2=7.78 U_RL3=9.42 U_BLOCK=12.71 engine_chosen=BLOCK final_chosen=BL0C  
K rate=0kbps reason=GAME_ENGINE_DECISION rep=0.69 round=4 snort=True pps=290.33  
BLOCK_RULE_INSTALLED src=10.0.0.4 dst=10.0.0.1 in_port=4 proto=ICMP spoofed=False idle timeout=30  
GAME DECISION src=10.0.0.4 dst=10.0.0.1 proto=ICMP in_port=4 spoofed=False NORMAL=0.00 PROBE=0.06 FLOOD_LOW=0.15 FLOOD_HIGH=0.64 IP_  
SPOOFING=0.01 SPOOFED_FLOOD=0.15 U_ALLOW=-11.33 U_RL1=2.36 U_RL2=7.72 U_RL3=9.41 U_BLOCK=11.82 engine_chosen=BLOCK final_chosen=BL0C  
K rate=0kbps reason=GAME_ENGINE_DECISION rep=0.66 round=4 snort=True pps=207.33  
RATE LIMIT APPLIED strategy=RL_2 rate=1024kbps src=10.0.0.1 dst=10.0.0.4 in_port=1 proto=ICMP spoofed=False  
GAME DECISION src=10.0.0.1 dst=10.0.0.4 proto=ICMP in_port=1 spoofed=False NORMAL=0.10 PROBE=0.29 FLOOD_LOW=0.24 FLOOD_HIGH=0.23 IP_  
SPOOFING=0.05 SPOOFED_FLOOD=0.10 U_ALLOW=-4.23 U_RL1=4.67 U_RL2=6.87 U_RL3=6.07 U_BLOCK=5.07 engine_chosen=RL_2 final_chosen=RL_2 ra  
te=1024kbps reason=GAME_ENGINE_DECISION rep=0.73 round=5 snort=True pps=0.00  
BLOCK_RULE_INSTALLED src=10.0.0.4 dst=10.0.0.1 in_port=1 proto=ICMP spoofed=False idle timeout=30  
GAME DECISION src=10.0.0.4 dst=10.0.0.1 proto=ICMP in_port=1 spoofed=False NORMAL=0.00 PROBE=0.14 FLOOD_LOW=0.22 FLOOD_HIGH=0.47 IP_  
SPOOFING=0.02 SPOOFED_FLOOD=0.15 U_ALLOW=-9.45 U_RL1=3.38 U_RL2=7.81 U_RL3=8.42 U_BLOCK=10.49 engine_chosen=BLOCK final_chosen=BLOCK  
rate=0kbps reason=GAME_ENGINE_DECISION rep=0.76 round=5 snort=True pps=73.67  
RATE LIMIT APPLIED strategy=RL_2 rate=1024kbps src=10.0.0.1 dst=10.0.0.4 in_port=1 proto=ICMP spoofed=False  
SPOOFING=0.05 SPOOFED_FLOOD=0.10 U_ALLOW=-4.23 U_RL1=4.67 U_RL2=6.87 U_RL3=6.07 U_BLOCK=5.07 engine_chosen=RL_2 final_chosen=RL_2 ra  
te=1024kbps reason=GAME_ENGINE_DECISION rep=0.77 round=6 snort=True pps=0.00
```

Figure 5.5 Real-Time Monitoring and Adaptive Mitigation Strategy Enforcement during ICMP Flood Attack.

Figure (5.5) illustrates the real-time monitoring and defensive decision-making process within the Software-Defined Networking (SDN) environment during the execution of an ICMP Flood attack scenario.

The logs demonstrate how the framework continuously analyzes network traffic behavior using several security indicators, including:

Flood probability

Spoofing probability

Packets Per Second (PPS)

Snort IDS alert status

Based on these observations, the Game Theory Engine calculates the utility values of multiple defense strategies in order to select the most suitable mitigation action.

The results show that the framework dynamically applies several graduated defensive mechanisms, including:

Rate-limiting policies

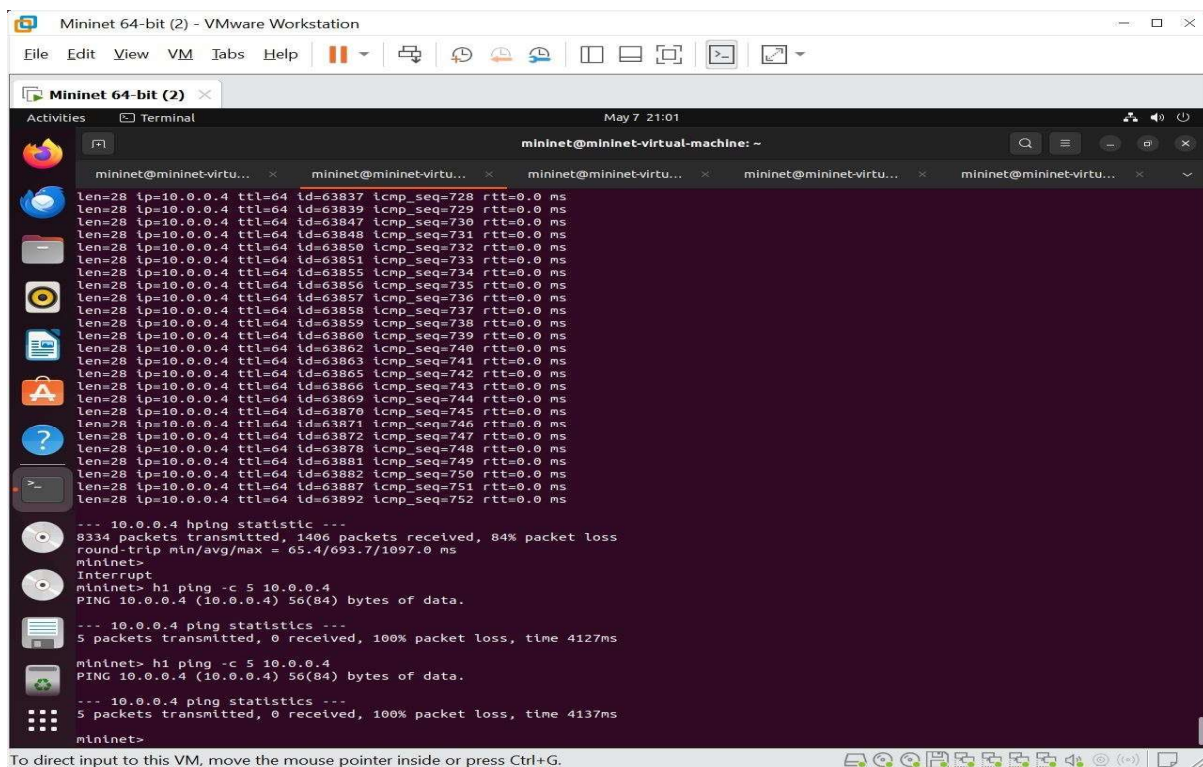
RL_1, RL_2, and RL_3 mitigation strategies

Installation of blocking rules (BLOCK_RULE_INSTALLED)

Reduction of malicious traffic rates

The logs further demonstrate that the framework continuously adapts its decisions according to attack severity and attacker behavior, which improves network resilience and minimizes the impact of flooding attacks within the SDN environment.

5.6 Packet Loss and Connectivity Impact



```
Mininet 64-bit (2) - VMware Workstation
File Edit View VM Tabs Help
Mininet 64-bit (2)
Activities Terminal May 7 21:01
mininet@mininet-virtual-machine: ~
mininet@mininet-virtu... x mininet@mininet-virtu... x mininet@mininet-virtu... x mininet@mininet-virtu... x mininet@mininet-virtu... x
len=28 ip=10.0.0.4 ttl=64 id=63837 icmp_seq=728 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63839 icmp_seq=729 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63847 icmp_seq=730 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63848 icmp_seq=731 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63850 icmp_seq=732 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63851 icmp_seq=733 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63855 icmp_seq=734 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63856 icmp_seq=735 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63857 icmp_seq=736 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63858 icmp_seq=737 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63859 icmp_seq=738 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63860 icmp_seq=739 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63862 icmp_seq=740 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63863 icmp_seq=741 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63865 icmp_seq=742 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63866 icmp_seq=743 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63869 icmp_seq=744 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63870 icmp_seq=745 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63871 icmp_seq=746 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63872 icmp_seq=747 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63878 icmp_seq=748 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63881 icmp_seq=749 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63882 icmp_seq=750 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63887 icmp_seq=751 rtt=0.0 ms
len=28 ip=10.0.0.4 ttl=64 id=63892 icmp_seq=752 rtt=0.0 ms
--- 10.0.0.4 hping statistic ---
8334 packets transmitted, 1406 packets received, 84% packet loss
round-trip min/avg/max = 65.4/693.7/1097.0 ms
mininet>
Interrupt
mininet> h1 ping -c 5 10.0.0.4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
--- 10.0.0.4 ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4127ms
mininet>
mininet> h1 ping -c 5 10.0.0.4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
--- 10.0.0.4 ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4137ms
mininet>
```

Figure 5.6 Impact of ICMP Flood Attack on Connectivity Quality and Packet Loss within the SDN Environment.

Figure (5.6) illustrates the practical connectivity test results within the Software-Defined Networking (SDN) environment during the execution of an ICMP Flood attack between the attacker host and the victim host.

The logs show the continuous transmission of ICMP packets along with several communication parameters, including:

Target IP address

ICMP sequence number

Round Trip Time (RTT)

Number of transmitted and received packets

The final statistics demonstrate a significant increase in packet loss rates, reaching:

84% packet loss during the intensive attack phase

100% packet loss during subsequent connectivity tests

These results clearly demonstrate the direct impact of flooding attacks on network performance and communication stability inside the SDN environment.

The logs also indicate that the mitigation and blocking mechanisms enforced by the Ryu Controller contributed to reducing the attacker's ability to maintain normal communication, validating the effectiveness of the implemented defense strategies.

These findings are used as practical indicators to evaluate the performance of the proposed security framework and its capability to mitigate denial-of-service attacks within Software-Defined Networking environments.

5.7 BLOCK Decision and OpenFlow Drop Rule

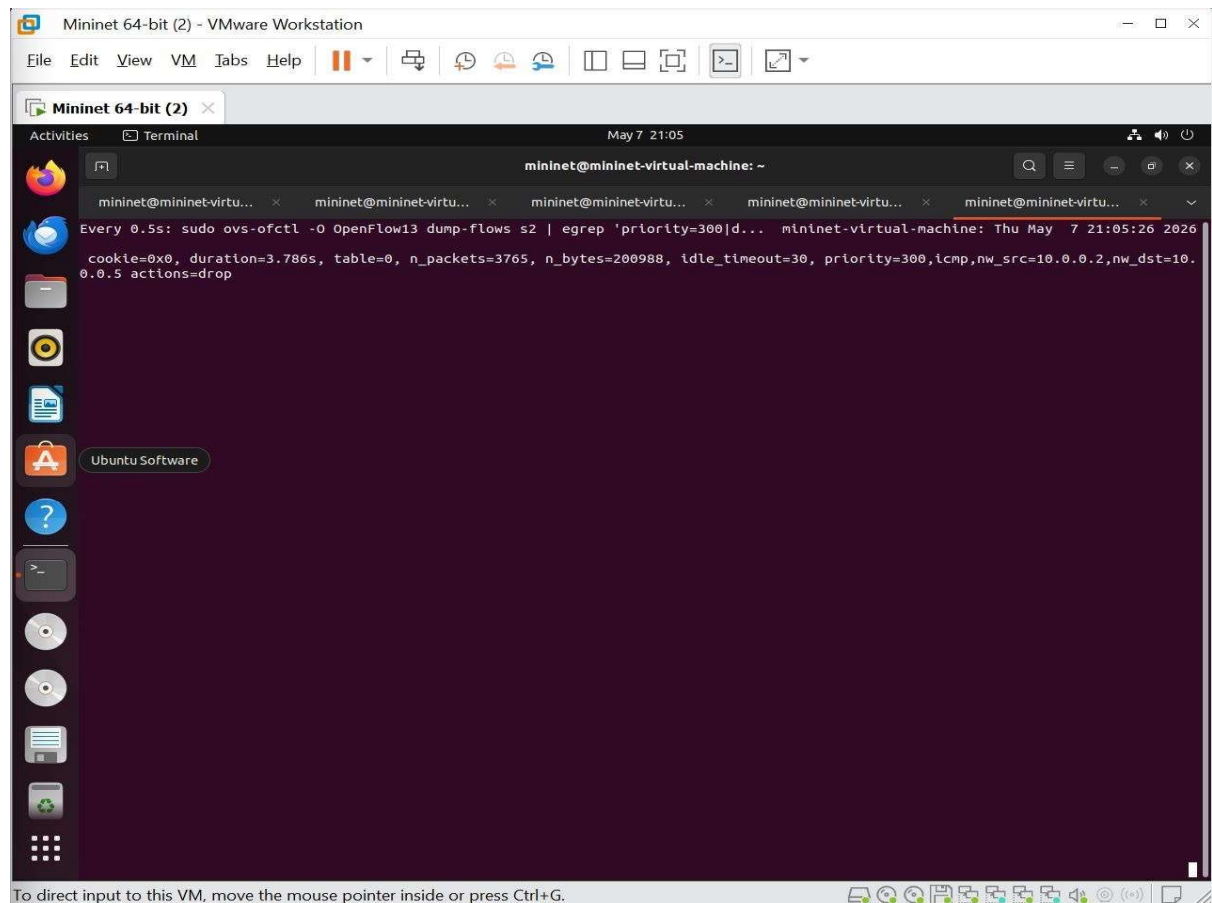


Figure 5.7 Verification of OpenFlow Drop Rule Installation within the Open vSwitch.

Figure (5.7) illustrates the verification process of the Ryu Controller successfully installing a drop rule inside the Open vSwitch using the OpenFlow 1.3 protocol during the mitigation phase of the ICMP Flood attack.

The logs show the execution of the command:

```
ovs-ofctl dump-flows
```

which is used to display the flow rules currently installed inside switch s2.

The results demonstrate the existence of a high-priority flow rule (priority=300) that blocks ICMP traffic originating from:

```
10.0.0.2
```

and targeting:

```
10.0.0.5
```

The log further shows the action:

```
actions=drop
```

which confirms that the controller successfully enforced the blocking policy directly inside the data plane rather than only generating a logical mitigation decision.

This result provides practical evidence of the successful integration between:

Snort IDS

The Game-Theoretic Decision Engine

Ryu Controller

OpenFlow Protocol

in implementing an adaptive security response capable of blocking malicious traffic within the Software-Defined Networking (SDN) environment.

5.8 IP Spoofing Attack Result

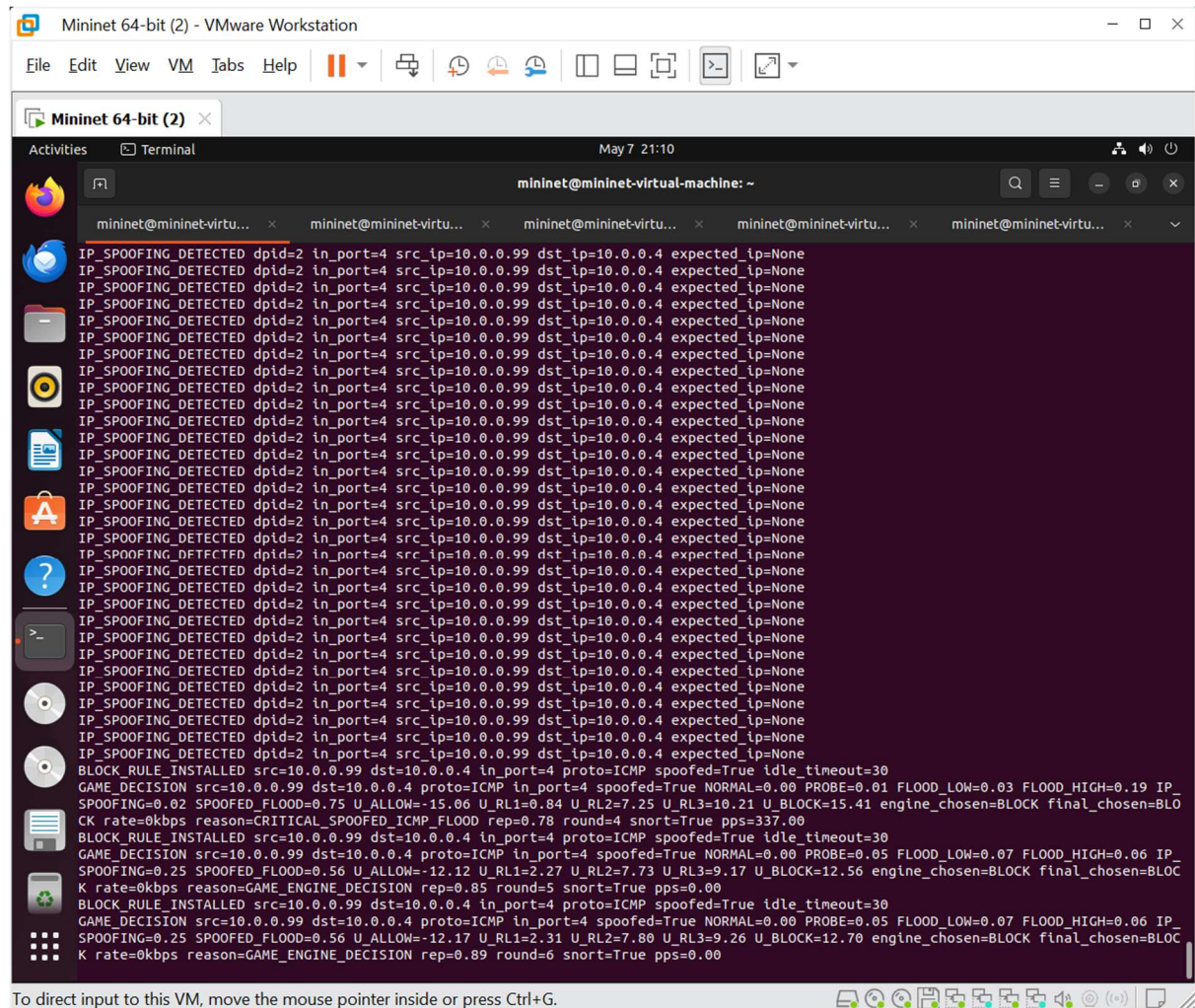


Figure 5.8 Detection of IP Spoofing Attack and Dynamic Blocking Enforcement within the SDN Environment.

Figure (5.8) illustrates the detection results of an IP Spoofing attack within the Software-Defined Networking (SDN) environment during the generation of malicious traffic using a forged IP address.

The logs show that the system successfully detected suspicious packets through repeated messages labeled:

IP SPOOFING DETECTED

The spoofed traffic originated from:

10.0.0.99

and targeted the victim host:

10.0.0.4

The results demonstrate that the framework compared the expected source identity with the actual packet source, enabling real-time detection of spoofed traffic behavior.

The logs further show that the analysis engine activated the status:

spoofed=True

while calculating:

Spoofing probability:

SPOOFING=0.25

Spoofed flood probability:

SPOOFED_FLOOD=0.56

Based on these indicators, the Game-Theoretic Decision Engine evaluated the attack severity and selected:

BLOCK

as the most appropriate mitigation strategy.

The screenshot also demonstrates the installation of blocking rules inside the OpenFlow switches through:

BLOCK_RULE_INSTALLED

which prevents the continuation of spoofed malicious traffic inside the network.

These findings confirm the capability of the proposed framework to:

Detect IP Spoofing attacks

Dynamically analyze threat severity

Automatically enforce blocking actions

Protect the SDN environment against spoofing-based attacks

The figure also validates the successful integration between:

Snort IDS

Ryu Controller

Game Theory Engine

OpenFlow-based mitigation

in implementing an intelligent and adaptive security response against network-based threats.

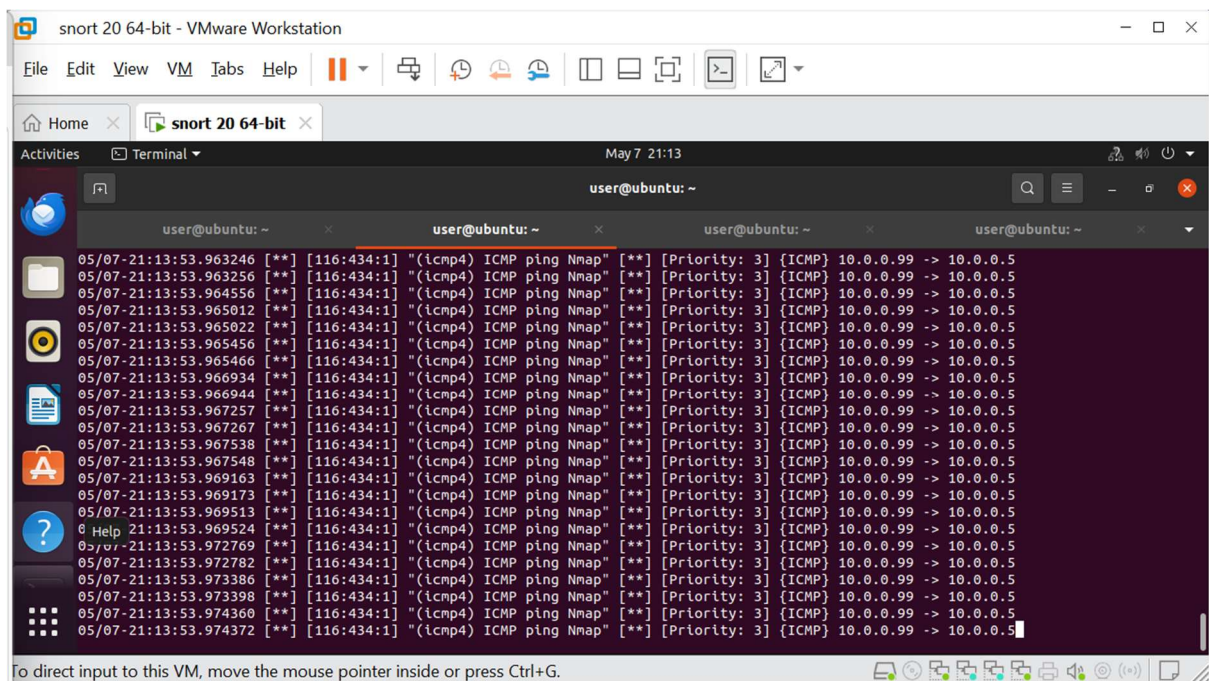


Figure 5.9 : Detection of Suspicious ICMP Traffic by Snort IDS within the SDN Environment.

Figure (5.9) illustrates the detection results of suspicious ICMP traffic by the Snort Intrusion Detection System (IDS) during the execution of an ICMP-based attack within the Software-Defined Networking (SDN) environment.

The logs show that Snort generated repeated alerts associated with the signature:

ICMP ping Nmap

which is commonly used to identify network scanning activities or ICMP flooding attacks.

The results also indicate that the system classified the attack with:

Priority: 3

while recording important network communication details, including:

Source IP address:

10.0.0.99

Destination IP address:

10.0.0.5

Protocol type:

ICMP

The logs demonstrate that a large number of suspicious packets were detected within a short period of time, indicating abnormal and potentially malicious traffic behavior inside the network.

These findings confirm the capability of Snort IDS to:

Monitor network traffic

Analyze packets in real time

Detect malicious activities

Generate accurate security alerts

This stage represents the foundational detection layer of the proposed security framework before forwarding alerts to the Ryu Controller for mitigation decisions using the Game-Theoretic Engine.

5.9 Evaluation Summary

The experimental results confirm that the proposed framework achieved the main project objectives. The system successfully mirrored traffic from the SDN switches to the Snort VM, detected ICMP-based

suspicious behavior, forwarded alerts to the Ryu controller, and applied adaptive mitigation decisions using the game-theoretic engine. The results also show that the framework can distinguish between normal traffic, suspicious traffic, high-rate flooding, persistent attacks, and spoofed traffic. Therefore, the system provides a practical proof-of-concept for adaptive SDN security using game theory.

6 Chapter Six: Conclusion

This project presented the design and development of an adaptive security framework for Software-Defined Networking (SDN) environments using game-theoretic decision making. The work was motivated by the fact that SDN provides centralized programmability and flexible network control, but at the same time exposes the controller and network resources to serious security threats, especially DDoS-related flooding attacks and spoofing-based traffic.

The project began by explaining the theoretical foundation required to understand the proposed framework. The theoretical part covered Software-Defined Networking, SDN architecture, OpenFlow, Open vSwitch traffic mirroring, Ryu Controller, Snort IDS, DDoS attacks, IP spoofing, and attacker–defender interaction using game theory. These concepts formed the scientific basis for building a security solution that connects traffic inspection, controller-side analysis, strategic decision-making, and mitigation enforcement.

The working environment was prepared using Mininet, Open vSwitch, Ryu Controller, Snort IDS, and a Python-based alert forwarding agent. Mininet was used to emulate the SDN topology, while Open vSwitch was used to forward traffic and support traffic mirroring. Snort IDS was placed on a separate virtual machine to inspect mirrored traffic, and the alert forwarding agent was used to transfer Snort alerts to the controller side. This design allowed the framework to separate traffic forwarding, traffic inspection, and mitigation decision-making while still keeping them connected in one closed-loop workflow.

The proposed solution was built around a game-theoretic decision engine. Instead of applying one fixed response to all suspicious traffic, the controller evaluates several observations before selecting a mitigation action. These observations include packet rate, Snort alert status, packet loss, RTT, spoofing indicators, repeated suspicious behavior, and attacker count. The `attacker_count` feature is particularly important because it supports the distinction between a single-source DoS-style flood and a multi-source DDoS-oriented flood. As a result, the framework can evaluate whether suspicious traffic is generated by one source or by multiple sources targeting the same victim.

The mitigation process in the proposed framework is adaptive and graduated. The controller may select ALLOW for normal traffic, RL_1 for low suspicious behavior, RL_2 for confirmed suspicious traffic, RL_3 for severe or distributed flooding behavior, and BLOCK for spoofed, persistent, or highly severe malicious traffic. The rate-limiting decisions are enforced through OpenFlow meter-based rules, while the BLOCK decision is enforced using

a high-priority drop rule. This shows that the proposed framework does not only detect suspicious traffic, but also translates the decision into an operational response inside the SDN data plane.

The evaluation part of the project demonstrated the feasibility of combining SDN programmability, Snort-based intrusion detection, alert forwarding, and game-theoretic reasoning. The results showed that the framework can distinguish between normal traffic, suspicious ICMP activity, high-rate flooding behavior, persistent attack behavior, and spoofed traffic. The DDoS-oriented design was strengthened by including the number of suspicious sources as part of the observation model, which provides a clearer basis for differentiating DoS and DDoS behavior in the proposed framework.

Overall, the project achieved its main objective by designing an integrated SDN security framework that supports adaptive DDoS-oriented mitigation. The strength of the project lies in connecting multiple layers together: Open vSwitch mirrors traffic, Snort detects suspicious behavior, the alert agent forwards IDS evidence, the Ryu controller evaluates the network state, the game-theoretic model selects the best response, and OpenFlow enforces the selected mitigation decision. This integration makes the project different from detection-only approaches because it links detection with decision-making and enforcement.

However, the project also has limitations. The implementation was conducted in an emulated academic environment rather than a real production network. In addition, the experimental scenarios focused mainly on ICMP-based flooding and spoofing-related behavior. Therefore, the results should be understood as a proof-of-concept that validates the feasibility of the proposed design, not as a complete production-ready security system. Larger networks, more diverse traffic patterns, and real-time performance measurements would be required before applying the framework in an operational environment.

Future work can extend this project in several directions. First, the framework can be tested using larger SDN topologies with more switches, hosts, and attack sources to evaluate scalability. Second, additional DDoS attack types can be included, such as TCP SYN flooding, UDP flooding, HTTP flooding, and low-rate DDoS attacks. Third, the payoff matrix can be refined using experimental measurements, expert evaluation, or historical traffic datasets instead of relying only on manually selected utility values. Fourth, the detection model can be improved by combining Snort alerts with flow statistics, machine

learning, or reputation-based scoring. Finally, future work may evaluate controller overhead, mitigation response time, false positives, packet loss, RTT, and the quality of legitimate traffic under different attack intensities.

In conclusion, this project provides an educational and practical proof-of-concept for enhancing SDN network security using game theory. It demonstrates how an SDN controller can move beyond static threshold-based protection and select proportional mitigation actions according to the evaluated threat severity. By integrating Snort IDS, OpenFlow enforcement, and game-theoretic decision-making, the proposed framework contributes to building a more adaptive and explainable approach for DDoS-oriented defense in SDN environments.

List of Terms and Abbreviations

Full Term	Abbreviation
Software-Defined Networking	SDN
Distributed Denial of Service	DDoS
Denial of Service	DoS
Intrusion Detection System	IDS
Open vSwitch	OVS
Virtual Machine	VM

Full Term	Abbreviation
Internet Control Message Protocol	ICMP
Internet Protocol	IP
Transmission Control Protocol	TCP
User Datagram Protocol	UDP
Round Trip Time	RTT
Packets Per Second	pps
Kilobits Per Second	kbps
Megabits Per Second	Mbps
Quality of Service	QoS
Central Processing Unit	CPU
Operating System	OS
Long Term Support	LTS
Rate Limiting Level 1	RL_1
Rate Limiting Level 2	RL_2
Rate Limiting Level 3	RL_3
Game Theory Engine	GTE
Software-Defined Network Controller	SDN Controller
OpenFlow Switch	OF Switch
Economic Denial of Sustainability	EDoS

References

- [1] F. Razvan and C. Mitica, "Enhancing network security through integration of game theory in software-defined networking framework," *International Journal of Information Security*, vol. 24, art. no. 100, 2025, doi: 10.1007/s10207-025-01012-4.
- [2] N. Gupta, S. Tanwar, and S. Badotra, "A novel sFlow based DDoS detection model in software defined networking," *International Journal of Applied Science and Engineering*, vol. 21, no. 2, art. no. 2023510, 2024, doi: 10.6703/IJASE.202406_21(2).005.

- [3] R. Wainwright, M. Bagheri, A. Salama, and R. Saatchi, "Software-Defined Networking Security Detection Strategies and Their Limitations with a Focus on Distributed Denial-of-Service for Small to Medium-Sized Enterprises," *Applied Sciences*, vol. 15, art. no. 12389, 2025, doi: 10.3390/app152312389.
- [4] A. Kachavimath and N. D. G., "Hybrid Approach for Detection and Mitigation of DDoS Attacks Using Multi-feature Selection, Unsupervised Learning, and Game Theory," *Journal of Telecommunications and Information Technology*, no. 4, pp. 20–32, 2025, doi: 10.26636/jtit.2025.4.2261.
- [5] M. M. Trigui, M. S. Islam, M. D. Gambo, M. S. Alam, and T. Helmy, "Resource-Aware Ensemble Framework for DDoS Detection in SDN via Novel Game-Theoretic Feature Selection," *IEEE Access*, vol. 14, pp. 1666–1676, 2026, doi: 10.1109/ACCESS.2025.3618519.
- [6] A. Javadpour, F. Ja'fari, T. Taleb, and C. Benzaïd, "Detecting malicious nodes using game theory and reinforcement learning in software-defined networks," *International Journal of Information Security*, vol. 24, art. no. 117, 2025, doi: 10.1007/s10207-025-01026-y.
- [7] F. Z. Chowdhury, M. N. Adnan, M. M. Siddique, A. Parvez, and M. Y. I. B. Idris, "Advanced EDoS eye: A pragmatic model to mitigate economic denial of sustainability attack in cloud system applying dynamic game-based decision module," *International Journal of Innovative Research and Scientific Studies*, vol. 9, no. 2, pp. 35–52, 2026, doi: 10.53894/ijirss.v9i2.11243.
- [8] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling Innovation in Campus Networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008, doi: 10.1145/1355734.1355746.
- [9] Open Networking Foundation, "OpenFlow Switch Specification, Version 1.3.0," Open Networking Foundation, 2012.
- [10] D. Kreutz, F. M. V. Ramos, P. Verissimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-Defined Networking: A Comprehensive Survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015, doi: 10.1109/JPROC.2014.2371999.
- [11] B. Lantz, B. Heller, and N. McKeown, "A Network in a Laptop: Rapid Prototyping for Software-Defined Networks," in *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*, 2010, doi: 10.1145/1868447.1868466.
- [12] Ryu SDN Framework Community, "Ryu SDN Framework Documentation," Ryu Project, 2026. Accessed: May 27, 2026.
- [13] Mininet Project, "Mininet Walkthrough," Mininet Documentation, 2026. Accessed: May 27, 2026.
- [14] Open vSwitch Project, "Open vSwitch Documentation: ovs-actions," Open vSwitch Documentation, 2026. Accessed: May 27, 2026.
- [15] Open vSwitch Project, "Open vSwitch Documentation: Configuration and Port Mirroring," Open vSwitch Documentation, 2026. Accessed: May 27, 2026.

- [16] Cisco / Snort Team, "Snort 3 User Manual," Snort Documentation, 2024.
- [17] Cisco / Snort Team, "Snort 3 Rule Writing Guide," Snort Documentation, 2026. Accessed: May 27, 2026.
- [18] Scapy Project, "Scapy Documentation," Scapy Project Documentation, 2026. Accessed: May 27, 2026.
- [19] S. Sanfilippo, "hping3 Manual Page," Linux Manual Pages, 2026. Accessed: May 27, 2026.
- [20] Python Software Foundation, "Python 3.10 Documentation," Python Documentation, 2026. Accessed: May 27, 2026.
- [21] Canonical, "Ubuntu 22.04 LTS Release Notes," Ubuntu Documentation, 2026. Accessed: May 27, 2026.
- [22] Y. Su, L. Xu, and Y. Zhao, "A Comprehensive Survey of Distributed Denial of Service Detection and Mitigation Technologies in Software-Defined Network," *Electronics*, vol. 13, no. 4, art. no. 807, 2024, doi: 10.3390/electronics13040807.
- [23] L. F. Eliyan and R. Di Pietro, "DoS and DDoS Attacks in Software Defined Networks: A Survey of Existing Solutions and Research Challenges," *Future Generation Computer Systems*, 2021.
- [24] J. Singh and S. Behal, "Detection and Mitigation of DDoS Attacks in SDN: A Comprehensive Review, Research Challenges and Future Directions," *Computer Science Review*, 2020.
- [25] Q. Yan, F. R. Yu, Q. Gong, and J. Li, "Software-Defined Networking and Distributed Denial of Service Attacks in Cloud Computing Environments: A Survey, Some Research Issues, and Challenges," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 1, pp. 602–622, 2016, doi: 10.1109/COMST.2015.2487361.
- [26] K. Wang, Y. Fu, X. Duan, and T. Liu, "Detection and Mitigation of DDoS Attacks Based on Multi-dimensional Characteristics in SDN," *Scientific Reports*, vol. 14, art. no. 16421, 2024, doi: 10.1038/s41598-024-66907-z.
- [27] H. Polat, O. Polat, and A. Cetin, "Detecting DDoS Attacks in Software-Defined Networks Through Feature Selection Methods and Machine Learning Models," *Sustainability*, vol. 12, no. 3, art. no. 1035, 2020, doi: 10.3390/su12031035.
- [28] P. Manso, J. Moura, and C. Serrão, "SDN-Based Intrusion Detection System for Early Detection and Mitigation of DDoS Attacks," *Information*, vol. 10, no. 3, art. no. 106, 2019, doi: 10.3390/info10030106.
- [29] K. Giotis, C. Argyropoulos, G. Androulidakis, D. Kalogeras, and V. Maglaris, "Combining OpenFlow and sFlow for an Effective and Scalable Anomaly Detection and Mitigation Mechanism on SDN Environments," *Computer Networks*, vol. 62, pp. 122–136, 2014, doi: 10.1016/j.comnet.2013.10.014.
- [30] R. Swami, M. Dave, and V. Ranga, "Software-Defined Networking-Based DDoS Defense Mechanisms," *ACM Computing Surveys*, vol. 52, no. 2, 2019, doi: 10.1145/3301614.

